

Randomized vs. Deterministic? Practical Randomized Synchronous BFT in Expected Constant Time

Xufeng Zhang
Beijing Institute of Technology
Beijing, China
xufengzhang.crypto@gmail.com

Baohan Huang
Beijing Institute of Technology
Beijing, China
huangbaohan@bit.edu.cn

Sisi Duan*
Tsinghua University
Beijing, China
duansisi@tsinghua.edu.cn

Haibin Zhang*
Yangtze Delta Region Institute of Tsinghua University, Zhejiang
Jiaxing, Zhejiang, China
bchainzhang@aliyun.com

Abstract—Most practical synchronous Byzantine fault-tolerant (BFT) protocols, such as Sync HotStuff (S&P 2020), follow the convention of partially synchronous BFT and adopt a deterministic design. Indeed, while these protocols achieve $O(n)$ time complexity, they exhibit impressive performance in failure-free scenarios.

This paper challenges this conventional wisdom, showing that a randomized paradigm terminating in expected $O(1)$ time may well outperform prior ones even in the failure-free scenarios. Our framework reduces synchronous BFT to a new primitive called multi-valued Byzantine agreement with strong external validity (MBA-SEV). Inspired by the *external validity* property of multi-valued validated Byzantine agreement (MVBA), the additional validity properties allow us to build a BFT protocol where replicas agree on the hashes of the blocks. Our instantiation of the paradigm, Sonic, achieves $O(n)$ amortized message complexity per block proposal, expected $O(1)$ time, and enables a fast path of only two communication step.

Our evaluation results using up to 91 instances on Amazon EC2 show that the peak throughput of Sonic and P-Sonic (a pipelining variant of Sonic) is 2.24x-14.52x and 3.08x-24.25x that of Sync HotStuff, respectively.

Index Terms—Byzantine fault-tolerant (BFT), synchronous BFT, randomized BFT, Byzantine agreement.

I. INTRODUCTION

Byzantine fault-tolerant state machine replication (BFT) is recognized as a fundamental component for reliable distributed systems [1]–[20] and blockchains [21]–[30]. This work focuses on synchronous BFT, where there exists a known upper bound Δ for message transmission and processing time. Synchronous BFT is known to tolerate half of Byzantine

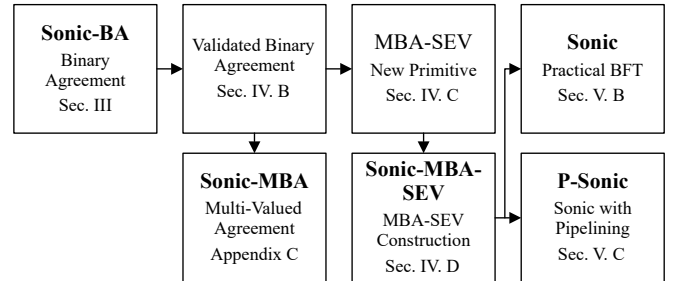


Fig. 1. Overview of this work.

failures [31], i.e., in a system with n replicas among which up to f are Byzantine, $n \geq 2f + 1$ is the lower bound.

Synchronous BFT protocols have long been of academic interest only. In recent years, several practical synchronous BFT protocols have been proposed [12], [32]–[34]. Almost all of them (e.g., Sync HotStuff [12] and its variants [32]–[34]) adopt a deterministic design and, in general, follow the paradigm of partially synchronous protocols. Namely, a designated leader *drives* the protocol in the normal-case operation and the leader is changed during the *view change*. In failure-free scenarios, these deterministic BFT protocols are known to achieve low latency and decent performance. The drawback is that the deterministic synchronous protocol must have $O(n)$ time complexity as the lower bound [31], [35], [36]. To circumvent the $O(n)$ time complexity barrier without weakening resilience, randomized synchronous BFT protocols with expected $O(1)$ time have been explored [29], [37]–[45]. However, most solutions are theoretical and have not been well explored in practical settings.

In contrast to synchronous BFT, partially synchronous BFT (where the bound Δ is unknown) and asynchronous BFT (where there does not exist such a bound) have been more extensively studied in a practical setting. A growing trend focuses on improving the throughput and scalability of the system by allowing multiple replicas to propose blocks concurrently [17], [18], [20], [46], [47]. Namely, by delivering a batch

Corresponding authors: Sisi Duan and Haibin Zhang.

This work is supported by the National Key R&D Program of China under 2022YFB2701500. Haibin Zhang is additionally supported in part by the National Natural Science Foundation of China under 62272043, Major Program of Shandong Provincial Natural Science Foundation for the Fundamental Research under ZR2022ZD03, Yangtze Delta Region Institute of Tsinghua University, Zhejiang (No. LZLX24F007). Sisi Duan is additionally supported in part by the National Natural Science Foundation of China under 92267203, Beijing Natural Science Foundation under M23015.

TABLE I

COMPARISON OF SYNCHRONOUS BFT PROTOCOLS. ALL PROTOCOLS ASSUME $n \geq 2f + 1$, WHERE n IS THE NUMBER OF PARTIES AND f IS THE NUMBER OF BYZANTINE FAILURES. THE TABLE IS DIVIDED INTO TWO PARTS: DETERMINISTIC (DET) AND RANDOMIZED (RND). FOR RANDOMIZED PROTOCOLS, WE CONSIDER THE EXPECTED TIME COMPLEXITY. THE AMORTIZED MESSAGE PER BLOCK DENOTES THE MESSAGE COMPLEXITY NEEDED TO DELIVER EACH BLOCK PROPOSAL. IN THE LATENCY COLUMN, Δ REFERS TO THE UPPER BOUND OF MESSAGE DELAY, AND δ MEANS THE ACTUAL MESSAGE DELAY ($\Delta \gg \delta$). IT IS CLEAR THAT LESS Δ NEEDED REPRESENTS LOWER LATENCY. IN THE ADVERSARY COLUMN, FOUR TYPES ARE INVOLVED, RANGING FROM THE WEAKEST TO THE STRONGEST: STATIC ADVERSARY, MILDLY ADAPTIVE ADVERSARY, (STANDARD) ADAPTIVE ADVERSARY, AND STRONGLY RUSHING ADAPTIVE ADVERSARY. RO DENOTES THE RANDOM ORACLE MODEL, AND CC REFERS TO THE COMMON COIN.

Protocol	Type	Time	Msg	Amortized Msg per Block	Latency (failure-free)	Adversary	Cryptographic Primitives
Sync HotStuff [12]	DET	$O(n)$	$O(n^2)$	$O(n^2)$	$2\Delta + \delta$	static / adaptive	PKI & hash
1 Δ -SMR [32]	DET	$O(n)$	$O(n^2)$	$O(n^2)$	$\Delta + 2\delta$	static / adaptive	PKI & hash
Sync-RR [34]	DET	$O(n)$	$O(n^2)$	$O(n^2)$	$2\Delta + O(\delta)$	static / adaptive	PKI & hash
OptSync [33]	DET	$O(n)$	$O(n^2)$	$O(n^2)$	2δ	static / adaptive	PKI & hash
PiLi [42]	RND	$O(1)$	$O(n^2)$	$O(n^2)$	$40\Delta \sim 65\Delta$	static	PKI & hash & RO
Streamlet [44]	RND	$O(1)$	$O(n^3)$	$O(n^3)$	12Δ	static	PKI & hash & RO
Dfinity [29], [45]	RND	$O(1)$	$O(n^2)$	$O(n^2)$	8Δ	mildly adaptive	PKI & hash & VRF
		$O(n)$	$O(n^3)$	$O(n^3)$	12Δ	(standard) adaptive	
ADDNR [41]	RND	$O(1)$	$O(n^2)$	$O(n^2)$	5Δ	static	PKI & hash & CC
					8Δ	strongly rushing adaptive	
Sonic & P-Sonic (this work)	RND	$O(1)$	$O(n^2)$	$O(n)$	2Δ	strongly rushing adaptive	PKI & hash & CC

of block proposals, the throughput of the system is improved. Notable examples include the asynchronous common subset (ACS) protocols [17], [18], [47], [48], which allow $O(n)$ block proposals to be delivered simultaneously. Such a paradigm, however, has never been explored for synchronous BFT.

In this paper, we propose a novel randomized synchronous BFT scheme that achieves expected $O(1)$ time complexity and outperforms previous deterministic ones even in failure-free scenarios. Our paradigm is inspired by the line of research on partially synchronous and asynchronous BFT, allowing multiple replicas to propose blocks simultaneously. Our protocol reduces the BFT to a new primitive called multi-valued Byzantine agreement with strong external validity (MBA-SEV). As the name suggests, we additionally add two stronger validity properties on top of conventional multi-valued Byzantine agreement (MBA), a primitive rarely used for practical BFT. The validity properties make our BFT protocol—in a surprisingly simple manner—use as input the hashes of block proposals, minimizing the concrete communication complexity. In comparison, almost all previous randomized protocols [29], [41]–[45] need to frequently exchange the bulk data (i.e., block proposals or *quorum certificates* where the size grows as n grows) and cannot be easily extended to the multi-proposer paradigm.

We summarize our technical pathway in Fig. 1. We first design a randomized binary agreement (BA) protocol, Sonic-BA, in synchronous settings under the strongly rushing adaptive adversary, having $O(n^2)$ message complexity and concretely fewer steps than the state-of-the-art BA [40]. Next, we propose Sonic-MBA-SEV, an MBA-SEV protocol built on a validated variant of Sonic-BA. Sonic-MBA-SEV provides a fast path as the key for our architecture to reach a latency of 2Δ under failure-free scenarios. Using Sonic-MBA-SEV, we build

our randomized synchronous BFT, Sonic, and its pipelined version, P-Sonic. As a by-product, we also design Sonic-MBA, a multi-valued Byzantine agreement (MBA) protocol having $O(n^2)$ message complexity and concretely fewer steps than the state-of-the-art MBA [41]. Each of the building blocks might be of independent interests.

We compare Sonic with recently proposed synchronous BFT solutions, as summarized in Table I. All protocols assume $n \geq 2f + 1$ and tolerate Byzantine failure. We categorize known protocols into two types: deterministic (DET) and randomized (RND). For time complexity, unlike deterministic protocols, we consider the expected time complexity for randomized protocols. There is a sharp contrast between these two types. For example, in terms of failure-free latency, deterministic Sync HotStuff [12] easily achieves *near-optimal* latency of $2\Delta + \delta$, while randomized Dfinity [29], [45] has a much higher latency. In comparison, Sonic achieves an even lower latency of 2Δ . It is worth noting that Sonic is the only one achieving $O(n)$ amortized message complexity per block proposal (i.e., the message complexity needed to deliver each block proposal). For the adversary model, deterministic protocols can naturally achieve adaptive security, while most previous randomized protocols assume a static adversary or a mildly adaptive adversary [49] (weaker than standard adaptive adversary). Sonic is secure against a strongly rushing adversary proposed in [41], which is stronger than the mildly and standard adaptive adversary model. For cryptographic primitives, all protocols require PKI and hash functions. Additionally, Sonic and ADDNR require common coins, which can be instantiated using threshold PRF or threshold signatures. Dfinity needs VRF. PiLi and Streamlet assume the random oracle model.

Our contributions are summarized below.

- We propose a randomized synchronous BFT paradigm with

expected $O(1)$ time complexity, beating previous deterministic ones even in failure-free scenarios. Based on a new primitive called multi-valued Byzantine agreement with strong external validity (MBA-SEV), our BFT protocol, Sonic, achieves $O(n)$ amortized message complexity per block proposal through batch delivery, and attains near-optimal latency of 2Δ under failure-free scenarios by a fast path.

- We present a synchronous binary agreement (BA) protocol, Sonic-BA, having optimal expected $O(1)$ time, $O(n^2)$ message, and concretely fewer steps than the state-of-the-art BA [40].
- We build synchronous BFT systems called Sonic and P-Sonic. Via the evaluation using up to 91 replicas on Amazon EC2, we show that for all metrics, Sonic consistently outperforms Sync HotStuff, and for $f = 1$ to $f = 30$, P-Sonic achieves much higher throughput than Sync HotStuff and Sonic. The peak throughput of Sonic is 2.24x-14.52x that of Sync HotStuff. For $f = 1$ to $f = 30$, the peak throughput of P-Sonic is 3.08x-24.25x that of Sync HotStuff.

II. SYSTEM MODEL, ASSUMPTIONS AND DEFINITIONS

A. System Model

We consider a system with n replicas, $\{p_0, \dots, p_{n-1}\}$, among which at most f are Byzantine (i.e., arbitrary failures). Non-Byzantine replicas are correct and we assume that they are always online. This follows the conventional BFT model. We assume authenticated, point-to-point channels between each pair of replicas. We assume a synchronous network and $n \geq 2f + 1$ ¹. For simplicity, we may assume $n = 2f + 1$. We let the upper bound for message transmission and processing delay be Δ . We set a synchronous start in our protocols.

We follow a conventional definition of the (communication) steps [52], which consists of receiving a message from another process, executing local computation, and sending a message to a certain process. Some agreement protocols run in rounds, each consisting of several steps. The total number of steps for a protocol is the sum of steps for all rounds it needs (perhaps plus some single steps required).

We consider *strongly rushing adaptive adversary* [41], in the sense that the adversary is both *adaptive* (where the adversary can choose the set of corrupted replicas at any moment during the execution of the protocol based on the information accumulated, but once a replica is corrupted, it cannot revert to a correct replica) and *strongly rushing* (where if the adversary chooses to corrupt p_i at some step, it controls which other correct replicas that p_i sends messages to and what messages that p_i sends to them in that step). Note that the strongly rushing adaptive adversary is stronger than the standard rushing adaptive adversary [53] and the mildly adaptive adversary [49]².

¹In comparison, in a partially synchronous or asynchronous network, $n \geq 3f + 1$ is optimal [50], [51].

²Mildly adaptive adversary can corrupt replicas adaptively but corruptions need $\Theta(\Delta)$ time to be effective.

In this work, we focus on the design and evaluation of the BFT protocols. For simplicity, we assume that all correct replicas remain online and therefore omit the recovery mechanism. A concise sketch of how to integrate replica recovery and state-transfer for missing messages is presented in Appendix G.

B. Primitives

Byzantine fault-tolerant (BFT). This paper mainly focuses on Byzantine fault-tolerant state machine replication (BFT) protocols. The BFT abstraction is specified by *bft-propose* and *bft-deliver*. Transactions are *bft-proposed* by clients (omitted for simplicity in this paper) to a certain replica simply by sending the transactions to the target replica. Every replica may *bft-propose* a block, i.e., a batch of transactions. All replicas interact to reach an agreement on the blocks and *bft-deliver* them. For the synchronous BFT protocol, $n \geq 2f + 1$ is needed. A secure BFT protocol achieves the following properties:

- **Safety:** If a correct replica *bft-delivers* a block b_1 before another block b_2 , then no correct replica *bft-delivers* b_2 before *bft-delivering* b_1 .
- **Liveness:** If a block b is *bft-proposed* to all replicas, then all correct replicas eventually *bft-deliver* b .

The BFT abstraction does not assign a particular order to some block. In practice, each block is often associated with a number, e.g., height. In this paper, we use *epoch* to denote the order. Namely, the protocol proceeds in epochs and at most one block is *bft-delivered* in each epoch.

Multi-valued Byzantine agreement (MBA). An MBA abstraction is specified by *mba-propose* and *mba-decide*. Each replica proposes a value v where $v \in \{0, 1\}^*$ and correct replicas will decide on some value. MBA should satisfy the following properties:

- **Validity:** If all replicas *mba-propose* v , then any correct replica that terminates *mba-decides* v .
- **Agreement:** If a correct replica *mba-decides* v , then any correct replica that terminates *mba-decides* v .
- **Termination:** Every correct replica eventually *mba-decides* some value.
- **Integrity:** No correct replica *mba-decides* twice.

Binary agreement (BA). A BA abstraction is a *binary* version of MBA. BA is specified by *ba-propose* and *ba-decide*. Each replica proposes an initial binary value v where $v \in \{0, 1\}$ and replicas will decide on some value. A BA primitive is the same in synchronous setting and asynchronous setting. The asynchronous version is also known as the asynchronous binary agreement (ABA). BA should satisfy the following properties:

- **Validity:** If all correct replicas *ba-propose* v , then any correct replica that terminates *ba-decides* v .
- **Agreement:** If a correct replica *ba-decides* v , then any correct replica that terminates *ba-decides* v .
- **Termination:** Every correct replica eventually *ba-decides* some value.

- **Integrity:** No correct replica *ba-decides* twice.

In most BA protocols, in addition to the *ba-propose* and *ba-decide* events, every replica still participates in the protocol after it decides, but simply skips the step of deciding some value. We say that a replica *terminates* the protocol if it no longer participates in the protocol.

For randomized BA protocols (which are built from common coins), a *good-case-coin-free* property [54], [55] is also defined, as shown below.

- **Good-case-coin-free:** If all correct replicas *ba-propose* the same value, then all correct replicas *ba-decide* without querying the common coin.

Since replicas do not have to query the common coin when all correct replicas *ba-propose* the same value, the protocol is more efficient in practice.

Validated binary agreement (VBA). Cachin, Kursawe, Petzold, and Shoup (CKPS) [48] define a *binary validated agreement* as a special case of multi-valued validated Byzantine agreement (MVBA) proposed in their work. As we consider only validated binary agreement, we define validated binary agreement (VBA) as a different primitive as MVBA in this work. In VBA, each replica adds a global predicate Q known to all parties such that $Q(v, \pi)$ holds and adds a proof π to the proposed value v . In practice, such a validation proof can be constructed using, e.g., threshold signatures.

In this work, we *convert* our BA protocol into VBA. We then build MBA from VBA, utilizing the global predicate $Q(v, \pi)$ for value $v = 1$. We do not need to check the predicate for $v = 0$. Therefore, we relax the definitions of CKPS and define the properties of VBA as follows:

- **External Validity:** Any correct replica that terminates and *vba-decides* 1 validated by π such that $Q(1, \pi)$ holds.
- **Validity:** If all correct replicas *vba-propose* v , then any correct replica that terminates *vba-decides* v .
- **Agreement:** If a correct replica *vba-decides* v , then any correct replica that terminates *vba-decides* v .
- **Termination:** Every correct replica eventually *vba-decides* some value.
- **Integrity:** No correct replica *vba-decides* twice.

C. Cryptographic Tools

In this paper, we denote the security parameter by λ . We assume a common coin oracle (protocol) for generating binary common coins and random leader election [48], [56]. In our experiments, we use threshold PRF [56] to instantiate the common coin oracle, which needs only one communication step with $O(n^2)$ message.

We use an $(f+1, n)$ threshold signature scheme π . A (t, n) threshold signature scheme [57], [58] consists of the following algorithms (*tgen*, *tsign*, *shareverify*, *tcombine*, *tverify*). The *tgen* algorithm outputs a system public key known to anyone and a vector of n secret keys. A partial signature signing algorithm *tsign* takes as input a message m and a secret key sk_i and outputs a partial signature π_i . A partial signature verification algorithm *shareverify* takes as input pk , a message

```

00 initialization
01  $vset_r \leftarrow \perp$                                 {value(s) to be agreed upon}
02  $vcount_r \leftarrow \perp$                           {number of matching values received}
03  $iv_r \leftarrow \perp$                              {proposed value for round  $r$ }
04 func ba-propose( $v_{input}$ )
05    $iv_1 \leftarrow v_{input}$ 
06   start round 1
07 round  $r$ 
08   start  $t_1 \leftarrow \Delta$ 
09    $\pi_i \leftarrow tsign(r, iv_r)$ 
10   send pre-vote $_r(iv_r, \pi_i)$  to all replicas
11   upon timeout( $t_1$ )
12     if there are  $f+1$  pre-vote $_r(v, \pi_j)$ 
13        $\pi \leftarrow tcomb(\pi_j, \dots)$ 
14       send main-vote $_r(v, \pi)$  to all replicas
15     else send main-vote $_r(\perp, \perp)$  to all replicas
16     start  $t_2 \leftarrow \Delta$ 
17   upon receiving main-vote $_r(v, \pi)$  s.t. tverify( $(r, v), \pi$ )
18     if timeout( $t_2$ ), discard the message
19      $vset_r \leftarrow vset_r \cup (v, \pi)$ 
20      $vcount_r(v) \leftarrow vcount_r(v) + 1$ 
21     {including message from  $p_i$  itself}
22   upon timeout( $t_2$ )
23     if  $r = 1$ ,  $coin_r \leftarrow 1$                                 {good-case-coin-free}
24     else  $coin_r \leftarrow Query()$ 
25     if  $\{(v, \pi)\} = vset_r$  and  $vcount_r(v) \geq f+1$ 
26       if  $v = coin_r$ 
27         ba-decide( $v$ )
28        $iv_{r+1} \leftarrow v$ 
29     else  $iv_{r+1} \leftarrow coin_r$ 
30      $r \leftarrow r + 1$ 

```

Fig. 2. Sonic-BA. $n = 2f + 1$.

m , and a partial signature π_i , and outputs a bit. A combining algorithm *tcombine* takes as input pk , a message m , and a set of t valid partial signatures, and outputs a signature π . A signature verification algorithm *tverify* takes as input pk , a message m , and a signature π , and outputs a bit. We require the conventional robustness and unforgeability properties for threshold signatures.

We use $H(m)$ to represent the hash digest of message m . We assume the collision resistance property of hash function. We use a conventional signature scheme consisting of three algorithms (*keygen*, *sign*, *verify*). The *keygen* algorithm outputs a pair of public/secret keys (pk, sk) . A signature signing algorithm *sign* takes as input a message m and a secret key sk and outputs a signature δ . A signature verification algorithm *verify* takes as input pk , a message m , a signature δ , and outputs a bit. We require the conventional unforgeability property for signatures.

In the pseudocode presented in this paper, we omit the public keys, secret keys, and key generation algorithms. For each received message that contains either a digital signature or a threshold signature, we may omit the verification algorithm and implicitly assume that signatures must be verified.

III. THE BINARY AGREEMENT—SONIC-BA

A. Overview

We present Sonic-BA, a synchronous binary agreement (BA) protocol. Sonic-BA is expected to terminate in 4 steps,

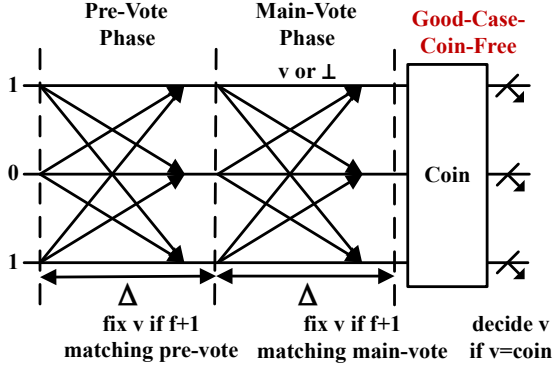


Fig. 3. The workflow of Sonic-BA.

and has $O(n^2)$ message complexity and $O(\lambda n^2)$ communication complexity. Compared to existing synchronous BA protocols, Sonic-BA achieves lower expected steps. Additionally, Sonic-BA achieves the good-case-coin-free property.

B. The BA Protocol

We illustrate our Sonic-BA protocol in Fig. 3 and show the pseudocode in Fig. 2. The protocol is round-based, starting from round 1. Each replica maintains three global parameters, each of which is used in every round of the protocol: $vset_r$, $vcount_r$, and iv_r . The $vset_r$ parameter is used to maintain the set of values each replica may agree on in each round. $vcount_r$ is used to count the number of messages received for each value in the $vset_r$. Finally, iv_r denotes the value the replica proposes in round r .

Each round consists of two phases: a pre-vote phase and a main-vote phase. Each phase involves one step with all-to-all communication. In the beginning of each round, replica p_i starts a timer t_1 and sends a $pre_vote_r(iv_r, \pi_i)$ message to all replicas, where iv_r is set to the input of the protocol (i.e., v_{input}) in round 1, and π_i is a partial signature for the value iv_r (as well as the protocol instance identifier and the round number).

After t_1 times out, p_i checks if it has received $f + 1$ $pre_vote_r(v)$ for the same v . If so, p_i combines these partial signatures into a threshold signature π , and sends an $main_vote_r(v, \pi)$ message to all replicas. Otherwise, p_i sends $main_vote_r(\perp, \perp)$ to all replicas. Then p_i starts timer t_2 . For each replica, upon receiving a $main_vote_r(v, \pi)$ message where π is a valid signature, the replica adds (v, π) to $vset_r$. After t_2 expires, the replica stops accepting any $main_vote_r()$ message. Replicas then query the common coin oracle and obtain $coin_r$. If p_i only has one value v in $vset_r$ and has received $f + 1$ $main_vote_r(v, \pi)$, p_i uses v as iv_{r+1} . p_i decides v if v equals the common coin. Otherwise, p_i uses the common coin as iv_{r+1} .

The value of the common coin is set to 1 in round 1. In this way, our protocol achieves the good-case-coin-free property [54], [55], which improves the performance.

We briefly explain the correctness of Sonic-BA. The unanimous validity comes from the fact that only the value with its signature π will be decided. For some value, if it is not proposed by a correct replica, a valid π of this value cannot be

formed. The agreement follows because there will be only one value whose signature π can be formed. Since the adversary cannot manipulate the input value used by replicas to enter the next round, the termination holds. We show the proof of Sonic-BA in Appendix A.

Terminating the protocol. In Sonic-BA, we rely on an additional terminating (exit) step where all replicas exchange their status after each replica decides [59]. To ensure that all correct replicas *ba-decide* and terminate, every replica still participates in the protocol after it decides v in round r . Meanwhile, each replica that decides sends a $terminate(v)$ message to all replicas. After receiving $f + 1$ $terminate(v)$ messages, any replica that has not yet *ba-decided* directly *ba-decides* v , sends $terminate(v)$ and then terminates.

Complexity. Sonic-BA achieves $O(n^2)$ messages, $O(\lambda n^2)$ communication complexity, and expected $O(1)$ time complexity. Against a static adversary, Sonic-BA terminates in only 3 steps in the optimistic case (where all replicas *ba-propose* 1) and is expected to terminate in 4 steps. Against a rushing adversary, Sonic-BA terminates in 3 steps in the optimistic case and is expected to terminate in 5 steps. Compared to existing the state-of-the-art BA protocol, Sonic-BA reduces the number of steps in the optimistic case and the expected number of steps. We provide more analysis in detail in Appendix B.

IV. TRANSFORMING SONIC-BA TO MBA WITH STRONG EXTERNAL VALIDITY

A. Overview

We are ready to present our new primitive called multi-valued Byzantine agreement with strong external validity (MBA-SEV). As mentioned in the introduction, MBA-SEV is used as a building block for our BFT paradigm, reducing communication costs and achieving lower latency compared with conventional synchronous BFT protocols.

We transform our BA protocol into an MBA-SEV protocol. Our technical pathway consists of two steps: transforming BA into Validated BA (VBA); reducing the problem of MBA-SEV to VBA. Following these two steps, we construct an MBA-SEV protocol named Sonic-MBA-SEV. The Sonic-MBA-SEV protocol achieves $O(n^2)$ message complexity and $O(Ln^2 + \lambda n^2)$ communication complexity (L denotes the message size). Sonic-MBA-SEV also enables a fast path allowing each replica to decide in only one single step. The fast path occurs when all replicas propose the same value.

As a by-product, our MBA-SEV protocol can be slightly modified to build a Multi-valued BA (MBA) protocol. We present the details in Appendix C, which might be of independent interest. Compared to existing MBA protocols, Sonic-MBA achieves fewer steps. Additionally, leveraging erasure coding proof systems [60], Sonic-MBA can achieve $O(1)$ time complexity, $O(n^2)$ message complexity, and $O(Ln + \lambda n^2)$ communication complexity.

B. Transforming BA into VBA

We first transform Sonic-BA into VBA. As defined in


```

00 initialization
01  $vset_r \leftarrow \perp$ 
02  $vcount_r \leftarrow \perp$ 
03  $iv_r \leftarrow \perp$ 
04  $Q(1, (h, \pi))$ : if  $tverify(h, \pi)$ , return 1; otherwise, return 0
                                     {predicate}

05 func  $vba-propose(v_{input}, (h, \pi_{input}))$ 
06  $iv_1 \leftarrow v_{input}$ 
07 start round 1
08 round  $r$ 
09 start  $t_1 \leftarrow \Delta$  ▷ Phase 1: Pre-vote
10  $\pi_i \leftarrow tsign(r, iv_r)$ 
11 if  $r = 1$  and  $iv_1 = 1$  {checking predicate  $Q$  in round 1 only}
12   send  $pre-vote_r(iv_r, \pi_i, (h, \pi_{input}))$  to all replicas
13 else send  $pre-vote_r(iv_r, \pi_i, \perp)$  to all replicas
14 upon receiving  $pre-vote_1(1, \pi_j, (h, \pi))$ 
15   if  $Q(1, (h, \pi))$  returns 0, discard the message
16 upon  $timeout(t_1)$ 
17   if there are  $f + 1$   $pre-vote_r(v, \pi_j, *)$ 
18      $\pi \leftarrow tcomb(\pi_j, \dots)$ 
19     send  $main-vote_r(v, \pi)$  to all replicas
20   else send  $main-vote_r(\perp, \perp)$  to all replicas
21   start  $t_2 \leftarrow \Delta$  ▷ Phase 2: Main-vote
22 upon receiving  $main-vote_r(v, \pi)$  s.t.  $tverify((r, v), \pi)$ 
23   if  $timeout(t_2)$ , discard the message
24    $vset_r \leftarrow vset_r \cup (v, \pi)$ 
25    $vcount_r(v) \leftarrow vcount_r(v) + 1$ 
26 upon  $timeout(t_2)$ 
27   if  $r = 1$ ,  $coin_r \leftarrow 1$  {good-case-coin-free}
28   else  $coin_r \leftarrow Query()$ 
29   if  $\{(v, \pi)\} = vset_r$  and  $vcount_r(v) \geq f + 1$ 
30     if  $v = coin_r$ 
31        $vba-decide(v)$ 
32      $iv_{r+1} \leftarrow v$ 
33   else  $iv_{r+1} \leftarrow coin_r$ 
34    $r \leftarrow r + 1$ 

```

Fig. 4. VBA version of Sonic-BA. In this figure, the major changes made on top of Sonic-BA are highlighted in blue.

Sec. II-B, VBA requires a global predicate Q known by all replicas. We may ignore the concrete definition of Q here and assume that given $Q(1, \pi)$, all replicas can verify whether $Q(1, \pi)$ holds.

The pseudocode of the VBA is shown in Fig. 4. To transform BA into VBA, we make two modifications on top of Sonic-BA. First, instead of proposing one value v_{input} , every replica $vba-proposes$ (v_{input}, π_{input}) , which is either $(1, \pi)$ or $(0, \perp)$. If a replica receives a vote for 1, it accepts the message only if $Q(1, \pi)$ holds. Second, instead of deciding one value v , every replica now $vba-decides$ either $(1, \pi)$ or $(0, \perp)$.

Correctness. The validity, agreement, termination, and integrity properties follow those of Sonic-BA. The external validity property follows as we require that every replica verifies whether $Q(1, \pi)$ holds upon receiving a $pre-vote_r(1, *)$. If a replica $vba-decides$ 1, it has received $f + 1$ $main-vote_r(v, *)$ messages. Thus, at least one correct replica has received $f + 1$ $pre-vote_r(1, *)$ messages, and $Q(1, \pi)$ holds. We show the proof of our VBA protocol in Appendix A.

C. Definition of MBA-SEV

Multi-valued Byzantine agreement with strong external

validity (MBA-SEV). We are now ready to define our MBA-SEV primitive. An MBA-SEV abstraction is specified by $mba-propose$ and $mba-decide$. Each replica $mba-proposes$ a value $v \in \{0, 1\}^*$ and correct replicas will $mba-decide$ some value. We require that every non- $\perp$ value v needs to be validated by some proof δ via a global predicate Q . A global predicate Q is known to all replicas such that v is valid only if $Q(v, \delta)$ holds. Note that if $v = \perp$, $Q(v, \perp)$ holds.

In addition to that, we further define a stronger validity constraint. In particular, if there exist two non- $\perp$ $mba-proposed$ values v and v' such that $v \neq v'$ and both values are validated, the two $mba-proposed$ values are called *equivocating* values. MBA-SEV should satisfy the following properties:

- **Unanimous validity:** If all correct replicas $mba-propose$ v and no replica can $mba-propose$ v' such that both $Q(v, \delta)$ and $Q(v', \delta')$ hold for some δ and δ' , then any correct replica that terminates $mba-decides$ v .
- **External validity:** Every correct replica that terminates $mba-decides$ v validated by δ such that $Q(v, \delta)$ holds.
- **Agreement:** If a correct replica $mba-decides$ v , then any correct replica that terminates $mba-decides$ v .
- **Non-intrusion:** If a correct replica $mba-decides$ v , v is $mba-proposed$ by at least one correct replica.
- **Termination:** Every correct replica eventually $mba-decides$ some value.
- **Integrity:** No correct replica $mba-decides$ twice.

Discussion. MBA-SEV is conceptually similar to both MBA and Multi-valued Validated Byzantine Agreement (MVBA) [48] (a multi-valued variant of VBA). For completeness, we provide the definition of MVBA in Appendix D. The external validity property follows that of MVBA and VBA. The agreement, termination, and integrity properties are identical to those of both MBA and MVBA. The non-intrusion property and the unanimous validity property are unique to the MBA-SEV primitive. The unanimous validity property is weaker than the validity property of MBA.

We briefly explain the motivation of the three new properties on top of MBA and how we use it to construct BFT protocols. To reduce BFT with full transactions to the agreement with compact hashes as input, we need the external validity property, similar to that in MVBA [48].

The unanimous validity property filters out the naive constructions that always output $\perp$. With the predicate being instantiated by the signature verification algorithm, the unanimous validity property guarantees that if all correct replicas propose the same validated hash, they will output a non- $\perp$ value and the BFT will make progress.

In addition, to ensure that at least one correct replica has the original blocks corresponding to the output hash of MBA-SEV, the non-intrusion property is needed.

D. Sonic-MBA-SEV: The MBA-SEV Protocol

The pseudocode of Sonic-MBA-SEV is shown in Fig. 5, and the workflow of Sonic-MBA-SEV is shown in Fig. 6.

We define the predicate Q such that $Q(v, \delta)$ holds if and only if δ is the valid signature of v (or, $v = \perp$ and $\delta = \perp$).

```

00 initialization
01  $v_{mba} \leftarrow \perp, \pi_{mba} \leftarrow \perp, \delta_{mba} \leftarrow \perp$ 
   {  $v_{mba}$  and  $\pi_{mba}$  are proofs of  $f + 1$  matching votes }
02  $V \leftarrow \emptyset, C \leftarrow \emptyset$ 
03  $Q(v, \delta)$ : if  $verify(v, \delta)$ , return 1; otherwise, return 0.
04 func  $mba-propose(v_{input}, \delta_{input})$ 
05   start  $t_1 \leftarrow \Delta$  ▷ Phase 1: Input
06    $\pi_i \leftarrow tsign(1, v_{input})$ 
07   send  $input(v_{input}, \pi_i, \delta_{input})$  to all replicas
08   upon receiving  $input(v_k, \pi_k, \delta_k)$  s.t.  $Q(v_k, \delta_k)$  holds
09      $V.add(v_k)$ 
10   upon timeout( $t_1$ )
11     if there are  $n$   $input(v, *, *)$  that  $v$  in  $V$  {fast-path}
12        $mba-decide(v)$  and send  $terminate(v)$  to all replicas
13       //still run the protocol but do not  $mba-decide$  again
14     if there are  $f + 1$   $input(v, \pi_j, \delta_j)$  for the same  $v$  in  $V$ 
15        $v_{mba} \leftarrow v, \delta_{mba} \leftarrow \delta_j, \pi_{mba} \leftarrow tcomb(\pi_j, \dots)$ 
16     if  $\pi_{mba} \neq \perp$  ▷ Phase 2: Distribute
17       send  $dist((v_{mba}, \delta_{mba}), \pi_{mba})$  to all replicas
18     else send  $dist(\perp, \perp)$  to all replicas
19     start  $t_2 \leftarrow \Delta$ 
20   upon receiving  $dist((v_k, \delta_k), \pi)$ 
21     if  $Q(v_k, \delta_k)$  holds and  $tverify((1, v_k), \pi)$ 
22        $C.add(v_k, \pi), V.add(v_k)$ 
23   upon timeout( $t_2$ )
24     if there are  $n - f$   $dist((v_k, \delta_k), \pi)$  and  $\{(v_k, \pi)\} = C$ 
       ▷ Phase 3: VBA
25        $vba-propose(1, (H(v_k), \pi))$  to VBA
26     else  $vba-propose(0, \perp)$  to VBA
27   if  $vba-decide(1, (h_k, \pi))$ 
28     obtain  $v_k$  s.t.  $(v_k, \pi) \in C$  and  $H(v_k) = h_k$ 
29      $mba-decide(v_k)$ 
30   else  $mba-decide(\perp)$ 
31   upon receiving  $f + 1$   $terminate(v)$  messages {fast-path}
32     if  $terminate()$  has not been sent
33       send  $terminate(v)$  to all replicas
34   upon receiving  $n$   $terminate(v)$  messages
35      $mba-decide(v)$  if not yet, terminate the protocol

```

Fig. 5. Sonic-MBA-SEV: MBA-SEV transformed from Sonic-BA. In this figure, the fast path is highlighted in blue.

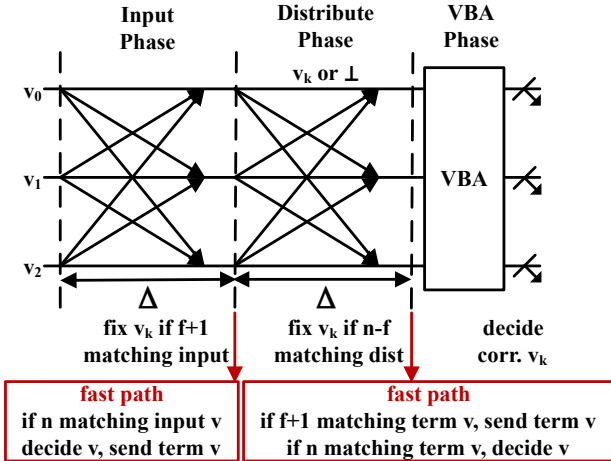


Fig. 6. The workflow of Sonic-MBA-SEV.

Each replica maintains a set V to keep track of validated values in $input()$ messages, and a set C to hold the (v, π) tuples from received $dist()$ messages.

Each replica $mba-propose$ s its input value v_{input} val-

idated by δ_{input} . It then starts a timer t_1 , and sends $input(v_{input}, \pi_i, \delta_{input})$ message to all replicas, where π_i is a partial signature on v_{input} . After receiving $input(v_k, \pi_k, \delta_k)$ message from another replica k , the replica checks if the predicate $Q(v_k, \delta_k)$ returns 1. If so, the replica adds v_k to its set V .

Fast path. After t_1 expires, there exists a fast path. Namely, each replica first checks if it has received n $input(v, *, *)$ messages for the same value v . If so, the replica $mba-decides$ v immediately, and sends $terminate(v)$ to all replicas.

Normal path. If the fast path is not triggered, each replica checks if there are at least $f + 1$ matching $input(v, \pi_j, \delta_j)$ messages and v is included in the set V . If so, the replica sets its global parameters v_{mba} and δ_{mba} as v_j and δ_j , respectively. Also, the replica combines the partial signatures into π_{mba} . Then, the replica sends $dist((v_{mba}, \delta_{mba}), \pi_{mba})$ to all replicas. Otherwise, it sends $dist(\perp, \perp)$ to all replicas. After sending $dist()$ messages, the replica starts a timer t_2 . Upon receiving $dist((v_k, \delta_k), \pi)$, the replica checks whether the predicate $Q(v_k, \delta_k)$ holds and the threshold signature π is valid. If so, the replica adds (v_k, π) to the set C and v_k to the set V .

After t_2 expires, each replica checks if there are $n - f$ matching $dist((v_k, \delta_k), \pi)$ messages, and (v_k, π) is the only element in C . If so, the replica $vba-propose$ s 1. Otherwise, the replica $vba-propose$ s 0. If the replica $vba-decides$ 1, it obtains the original tuple and $mba-decides$ the corresponding value v_k . Otherwise, if the replica $vba-decides$ 0, it $mba-decides$ $\perp$.

Termination. Throughout the protocol, replicas continue to collect $terminate()$ messages. If a replica receives $f + 1$ matching $terminate(v)$ messages and has not yet sent a $terminate()$ message, it also sends $terminate(v)$ to all replicas. After receiving n matching $terminate(v)$ messages, it terminates the protocol.

We briefly explain the correctness of Sonic-MBA-SEV. The unanimous validity comes from the fact that only the value with its signature π will be decided. For some value, if it is not proposed by a correct replica, a valid π of this value cannot be formed. The agreement follows because there will be only one non- $\perp$ value whose signature π can be formed. Since at least one correct replica is included in the intersection of the source of $f + 1$ $input()$ messages and $n - f$ $dist()$ messages, the non-intrusion holds. We show the proof of Sonic-MBA-SEV in Appendix E.

V. BUILDING SONIC SYNCHRONOUS BFT USING MBA-SEV

A. Overview

We are finally ready to present our BFT protocol Sonic that reduces BFT to our new primitive MBA-SEV. As mentioned in the introduction, our idea is to employ a multi-proposer paradigm and allow all replicas to propose new blocks. As illustrated in Fig. 8, Sonic proceeds as follows. In each epoch, each replica proposes a block. Via a random permutation, a random replica p_k is selected. Each replica then uses the hash

of the block proposed by p_k as the input to MBA-SEV. If a non- $\perp$ value h is decided, each replica then obtains the corresponding block and then *bft-delivers* the block.

One may wonder why not directly using MBA instead. Indeed, one can use MBA to replace MBA-SEV and agreement on the hash can still be reached. However, there is no guarantee that all correct replicas can obtain the corresponding block. Indeed, to ensure that at least one correct replica holds the block, we will need a "non-intrusion" property [61], i.e., if a replica decides some value, the value has been proposed by at least one correct replica. Unfortunately, to the best of our knowledge, no synchronous MBA can achieve such a property yet. Alternatively, each replica has to propose the block directly to MBA, making the paradigm less practical.

Accordingly, we define the non-intrusion property for our MBA-SEV primitive to make the communication-efficient paradigm possible. We are not aware of any way to provide a signature-free construction to build an MBA protocol with the non-intrusion property. Thus, we further define the unanimous validity and external validity properties for our MBA-SEV primitive. We also use unanimous validity to rule out naive constructions.

On top of the paradigm, we further provide a unique feature for Sonic: $O(n)$ block proposals are expected to be delivered in each epoch. Instead of agreeing on the block proposals by different replicas in each epoch, we take a different route: we deliver the historical blocks proposed by p_k . As illustrated in Fig. 9, each time $O(n)$ historical blocks proposed by p_k are agreed, and the input of MBA-SEV is still only one hash!

To summarize, with our Sonic-MBA-SEV instantiation, Sonic reaches expected $O(1)$ time complexity, as well as a latency of 2Δ under failure-free scenarios. Therefore, Sonic simultaneously achieves low latency and high throughput.

B. Synchronous BFT Paradigm Using MBA-SEV

We show the pseudocode of our synchronous BFT paradigm in Fig. 7 and demonstrate the workflow in Fig. 8. The protocol proceeds in epochs, and each epoch consists of three phases: propose, permutation, and agreement. In the propose phase, each replica proposes a block (or $\perp$ if the replica does not have any block remains to be proposed). In the permutation phase, a random replica p_k is selected. In the agreement phase, replicas agree on whether the blocks proposed by p_k can be safely delivered.

The unique feature of our protocol, as mentioned above, is the batch delivery mechanism that allows replicas to agree on a batch of blocks proposed by p_k . We now explain the mechanism in detail. Specifically, each replica p_i maintains a queue $M[j]$ to track the block proposals it has received from p_j (line 14). $M[j]$ thus holds all the undelivered block proposals by p_j . If p_k is selected during the permutation phase, the input of each replica to MBA-SEV is the head of the hash chain of $M[k]$ to aggregate all the blocks in $M[k]$ (lines 19-21). If a hash h_e is *mba-decided*, each replica will obtain the corresponding batch of block proposals and deliver the corresponding aggregated block (lines 26-30).

```

00 initialization
01  $C \leftarrow \emptyset$  {queue for received bft-proposed blocks}
02  $M \leftarrow [\emptyset]^n$  { $n$  queues, each stores blocks proposed by corresponding replica}
03 start epoch 1
04 upon receiving bft-proposed  $b_c$ 
05  $C.enqueue(b_c)$ 
06 epoch  $e$ 
07  $B_e \leftarrow \emptyset$  {aggregated block for epoch  $e$ }
08 if  $C \neq \emptyset$  ▷ Phase 1: Propose
09  $b \leftarrow C.dequeue()$  {get the first undelivered  $b$ }
10 send propose( $b$ ) to all replicas
11 else send propose( $\perp$ ) to all replicas
12 start  $t \leftarrow \Delta$ 
13 upon receiving propose( $b$ ) that  $b \neq \perp$  from  $p_j$ 
14  $M[j].enqueue(b)$ 
15 upon timeout( $t$ )
16 Choose a random permutation  $\Pi$  ▷ Phase 2: Permutation
17  $k \leftarrow \Pi()$ 
18 while  $M[k] \neq \emptyset$  ▷ Phase 3: Agreement
19  $b \leftarrow M[k].dequeue()$ 
20  $B_e.append(b)$ 
21  $h_e \leftarrow HashAppend(h_e, H(b))$ 
22 if  $B_e \neq \emptyset$ ,  $\delta_e \leftarrow sign(h_e)$ 
23 else  $h_e \leftarrow \perp$ ,  $\delta_e \leftarrow \perp$ 
24 mba-propose( $h_e, \delta_e$ ) to MBA-SEV
25 if mba-decide( $h'_e$ ) that  $h'_e \neq \perp$ 
26 if  $h'_e = h_e$ 
27 bft-deliver( $B_e$ )
28 else send request( $e, h'_e$ ) to all replicas
29 upon receiving response( $e, B_{e'}$ )
30 if  $h'_e = HashChain(B_{e'})$ 
31 bft-deliver( $B_{e'}$ )
32  $e \leftarrow e + 1$ 
33 upon receiving request( $e, h$ ) from  $p_j$ 
34 if  $B_e \neq \emptyset$  that has been delivered and  $h = HashChain(B_e)$ 
35 send response( $e, B_e$ ) to  $p_j$ 

```

Fig. 7. The Sonic protocol that reduces BFT to MBA-SEV.

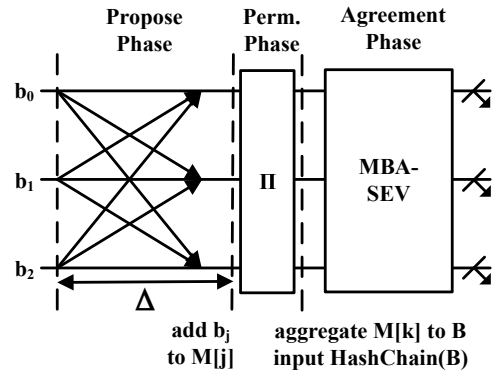


Fig. 8. The workflow of our BFT paradigm (for each epoch).

Indeed, the permutation phase can be executed in parallel with the propose phase. Doing so is correct due to the non-intrusion property of MBA-SEV.

We use Sonic-MBA-SEV to instantiate the MBA-SEV protocol in our BFT paradigm. We call the resulting instantiation Sonic.

An example. We illustrate the batch delivery mechanism in Fig. 9. In the figure, each row denotes the sequence of blocks

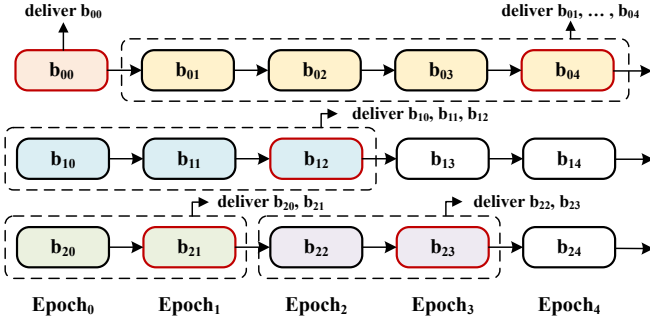


Fig. 9. Example of batch delivery mechanism of our BFT paradigm.

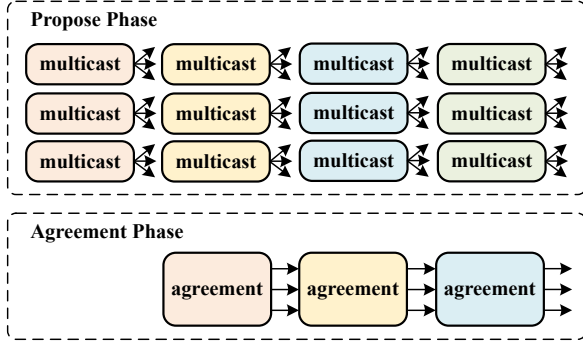


Fig. 10. Illustration of P-Sonic. Phases corresponding to the same epoch are highlighted in the same color. Multicast denotes *best-effort broadcast* [52], where each replica sends a message to all replicas.

proposed by each replica, i.e., $b_{00}, \dots, b_{04}$ are proposed by p_0 , while $b_{10}, \dots, b_{14}$ are proposed by p_1 . A rounded rectangle with red edges denotes that the corresponding replica is selected in the permutation phase. In epoch 0, p_0 is selected in the permutation phase, so b_{00} is *bft-delivered*. In epoch 1, p_2 is selected, and an aggregated block containing b_{20} and b_{21} is *bft-delivered*. Here, our protocol ensures that, once some hash value h is *mba-decided*, at least one correct replica holds a valid $M[k]$ such that the head of the hash chain of $M[k]$ is h . In this case, all replicas are able to obtain the aggregated block containing b_{20} and b_{21} , and then deliver it.

Complexities. Sonic achieves expected $O(1)$ time complexity and $O(n^2)$ message complexity. In each epoch, the probability that in the permutation phase, the chosen replica is correct is about $1/2$. Thus, the expected round number for delivering a block is two. Due to the parallelization of the propose phase and the permutation phase, in fast path, Sonic only needs two steps.

Our batch delivery mechanism allows a batch of proposals to be aggregated and *bft-delivered* in each epoch. In expectation, each replica will be selected once every n epochs. Thus, in each epoch, $O(n)$ block proposals will be delivered in expectation. Accordingly, the amortized message complexity per block proposal is $O(n^2)/n = O(n)$.

We show the proof of our BFT paradigm in Appendix F.

C. P-Sonic: Pipelining Sonic

Our Sonic protocol can be further extended to the pipelining model. Namely, new blocks can be proposed before the current

proposed block is delivered. We call the variant that supports pipelining P-Sonic.

To better describe P-Sonic, we merge the permutation phase into the propose phase. In P-Sonic, the propose phase can be de-coupled from the agreement phase. As shown in Fig. 10, replicas can directly enter new epochs by proposing new blocks, while the agreement phase of the current epoch is still running in parallel. However, when each replica delivers a block, it needs to wait for all prior epochs to finish before executing the corresponding transactions in the block.

Note that many partially synchronous [20], [62], [63] and asynchronous BFT protocols [46], [64] have validated that decoupling block proposals from agreement on the order can be used to improve the performance and even scalability of the system. Later, we show that our P-Sonic also validates that such a paradigm also works in a synchronous network. However, it becomes much trickier to tune the system parameters, e.g., the estimation of Δ becomes extremely challenging.

VI. IMPLEMENTATION AND EVALUATION

We compare Sonic and P-Sonic with Sync HotStuff [12], one of the most efficient synchronous BFT protocols. For randomized synchronous BFT protocols, while Dfinity [29], [45] is the only one that has been implemented, it has shown in [12] that Sync HotStuff strictly outperforms Dfinity. We thus omit performance comparison with Sonic, and consider Sync HotStuff the state-of-the-art synchronous BFT. To obtain a fair comparison, we implement Sonic, P-Sonic, and Sync HotStuff in a new Golang library. Our implementation involves around 10,000 LOC for the protocols and about 1,000 LOC for evaluation.

We use the Golang standard cryptography library *crypto* to implement the cryptographic operations. We use HMAC to implement authenticated channels and SHA256 to instantiate the hash function. We use ECDSA as the signature scheme. Namely, a set of $f + 1$ signatures are used as a certificate to instantiate the π scheme. To realize common coins and random permutations, we implement the threshold PRF [56] in our library. We use gRPC as the underlying communication library.

We evaluate the protocols on Amazon EC2 in the same region us-west-1 (Northern California), using up to 91 virtual machines (VMs). We believe this is closer to the synchronous environment. The network latency for a 1-bit message is less than 1 ms. The experiments were conducted on m5.xlarge instances, each with 4 virtual CPUs and 16 GB memory.

The experiments were conducted under different network sizes (denoted by f) and batch sizes (denoted by b). In all experiments, we used $n = 2f + 1$ replicas in total. The default transaction size is 50 bytes. For each experiment, we run the protocol five times and report the average latency and throughput.

The timers Δ of these protocols under the same network sizes f are set to the same value, following the convention of prior works [12], [33], [34]. Given each f , we tune the parameter Δ to a large enough value such that a block proposal

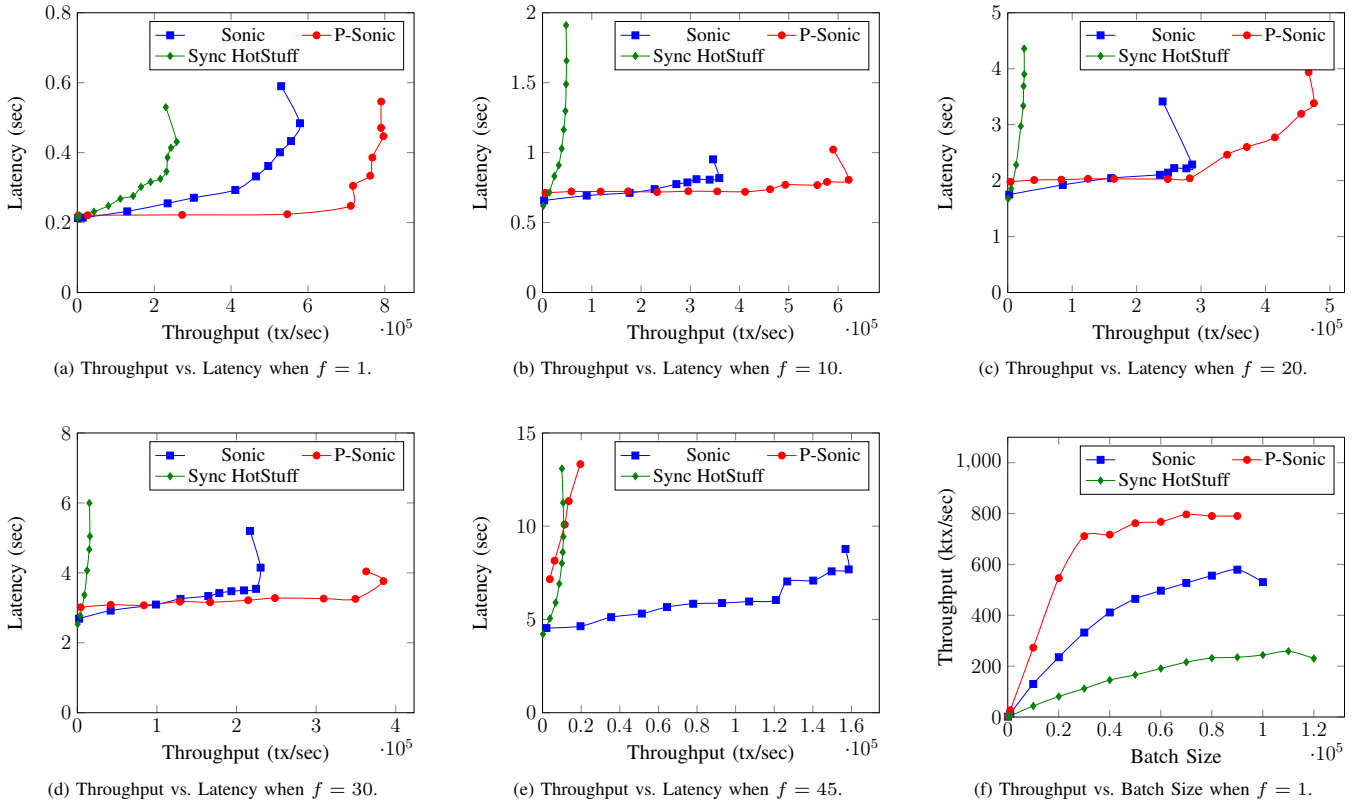


Fig. 11. Throughput vs. latency on m5.xlarge instances for $f = 1$ to $f = 45$.

with 10,000 transactions can be received by all replicas. Concretely, the timer is set to 100 ms when $f = 1$, 300 ms when $f = 10$, 800 ms when $f = 20$, 1,200 ms when $f = 30$, and 2,000 ms when $f = 45$.

In each experiment, we gradually increase the batch size until the throughput stops growing but latency continues to increase. When the batch size is large enough and when all replicas are actively sending proposals, there might be no guarantee in receiving the messages within the Δ time.

The main results are summarized as follows.

- For $f = 1$ to $f = 45$, Sonic consistently outperforms Sync HotStuff. As f grows, the performance gain of Sonic over Sync HotStuff also grows. The peak throughput of Sonic is 2.24x-14.52x that of Sync HotStuff.
- For $f = 1$ to $f = 30$, P-Sonic outperforms both Sonic and Sync HotStuff. The peak throughput of P-Sonic is 3.08x-24.25x that of Sync HotStuff and 1.37x-1.73x that of Sonic. However, P-Sonic fails to scale to $f = 45$ ($n = 91$) in our case, but Sonic and Sync HotStuff can. We believe this reveals the challenges of estimating Δ for synchronous protocols under the pipelining mode.
- Under failures, both Sonic and P-Sonic consistently outperform Sync HotStuff.

Throughput vs. latency. We show throughput vs. latency of Sonic, P-Sonic, and Sync HotStuff in Fig. 11a-Fig. 11e. We show throughput vs. batch size for $f = 1$ in Fig. 11f.

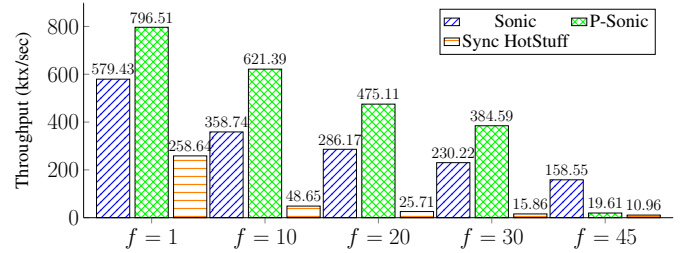


Fig. 12. Peak throughput of the protocols as f grows.

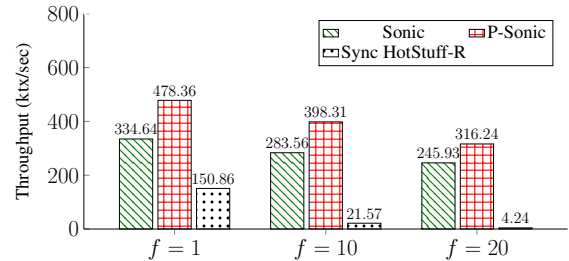


Fig. 13. Peak throughput of the protocols under f crash failures.

Sonic consistently outperforms Sync HotStuff for all f . This is mostly because Sonic allows each replica to *bft-deliver* $O(n)$ block proposals in each epoch. Meanwhile, due to batch delivery, as the network size grows, the throughput improvement of Sonic over Sync HotStuff also increases. Namely, the peak throughput of Sonic is 2.24x, 7.37x, 11.13x,

14.52x that of Sync HotStuff for $f=1, 10, 20, 30$, respectively.

P-Sonic outperforms both Sync HotStuff and Sonic for $f = 1$ to $f = 30$. However, when $f = 45$, Sonic outperforms P-Sonic, and the peak throughput of P-Sonic is only slightly higher than that of Sync HotStuff. We believe this is because the propose phase and the agreement phase are executed concurrently. There is no guarantee in receiving each block proposal within Δ time. In fact, even by slightly increasing Δ time for $f = 45$, timeout frequently occurs in our experiments, and the performance is still lower than that of Sonic.

Peak throughput. We show the peak throughput Sonic, P-Sonic, and Sync HotStuff in Fig. 12. The peak throughput of Sonic is consistently higher than that of Sync HotStuff. For instance, under $f = 1$, the peak throughput of Sonic is 2.24x that of Sync HotStuff. For $f = 30$, the peak throughput of Sonic is 14.52x that of Sync HotStuff. Additionally, the peak throughput of P-Sonic is 3.08x-24.25x that of Sync HotStuff and 1.37x-1.73x that of Sonic for $f = 1$ to $f = 30$. The results match those for throughput vs. latency.

Performance under crash failures. We report the peak throughput of the protocols under failures in Fig. 13. In each experiment, we randomly crash f replicas. As Sync HotStuff is leader-based, we evaluate the rotating leader mode and call it Sync HotStuff-R [34]. Namely, view change is triggered periodically. For both Sonic and Sync HotStuff-R, when a crashed replica is selected, the throughput becomes 0 in this epoch. As shown in the figure, the peak throughput of Sonic degrades 14.25%-42.25% under failures, and the throughput of P-Sonic degrades 33.44%-39.94%. The performance degradation is expected because if a crashed replica is selected in the permutation phase, no blocks can be *bft-delivered*. In contrast, the performance of Sync HotStuff-R degrades more significantly. This is mainly because Sync HotStuff-R involves a view change phase that is more expensive than the normal operations. In all the scenarios, Sonic and P-Sonic outperform Sync HotStuff-R.

Our evaluation focuses on crash failures because in a synchronous network, the performance under Byzantine failures will not be worse than that under crash failures. Specifically, the adversary can not delay the process of correct replicas, as each correct replica immediately proceeds to the next step after Δ time. The only thing the adversary can do is to not send some messages, similar to the crash failure scenarios.

VII. RELATED WORK

Synchronous BA. Byzantine agreement (BA) is a fundamental problem in distributed computing. In a synchronous network with authenticated channels, Dolev and Strong [35] propose a Byzantine broadcast protocol under $n > f$ with $O(n)$ time, and prove that the lower bound of the time complexity of the deterministic Byzantine broadcast protocol is $O(n)$. It has been shown by Fitzi [31] that the Dolev and Strong Byzantine broadcast protocol can be easily transformed into a BA protocol under $n > 2f$ with $O(n)$ time, having the same lower time bound as the deterministic Byzantine broad-

cast protocol. The state-of-the-art deterministic BA protocol is proposed by Momose and Ren [36], achieving optimal communication complexity. To circumvent the $O(n)$ time barrier, randomization has been explored [37]. Feldman and Micali [38] present a randomized BA protocol achieving $n > 3f$ with expected $O(1)$ time assuming private channels. In authenticated synchronous settings, Byzantine agreement can be achieved under $n > 2f$ with expected $O(1)$ time. Fitzi and Garay [39] introduce such a protocol using randomization with specific number-theoretic assumptions, while Katz and Koo [40] present the first constant-round protocol in this setting only assuming PKI and digital signatures. Abraham et al. [41] further optimize the expected number of steps. In comparison, Sonic achieves lower expected steps than the state-of-the-art BA protocols and enables a fast path, shown in Appendix B. Recently, Wang et. al. proposed a BA protocol in [65] achieves strict $O(n)$ messages, while Sonic-BA achieves $O(n)$ amortized messages. However, to achieve strict $O(n)$ messages, BA in [65] only considers $f < n/3$ and the static adversary. In contrast, Sonic-BA achieves optimal resilience, which is $f < n/2$, and adaptive security.

Deterministic synchronous BFT. The family of Sync HotStuff [12] and its variants [32]–[34], [66] exhibit low latency under failure-free scenarios. Under a correct leader, replicas deliver in $2\Delta + O(\delta)$ time in Sync HotStuff. For optimization, several works focus on reducing the latency under good case (when the leader is correct), while the others try to add optimistic responsiveness (when a portion of replicas are correct). For stable-leader protocols, Abraham et al. [32], [66] prove the $\Delta + 2\delta$ lower bound of concrete good-case latency and propose a protocol that achieves this latency. Shrestha et al. [33] propose an optimistically-responsive protocol with a fast path. This protocol achieves 2δ latency if the leader is correct and $f < n/4$. For rotating-leader protocols, Abraham et al. [34] present the $2\Delta + O(\delta)$ lower bound of good-case latency including both latency of delivering and view-change. Very recently, to improve the practicality of synchronous BFT protocols, Milošević et. al. [67] propose a rotating-leader BFT protocol that calibrates protocol-specific synchrony bounds, allowing much smaller timers (Δ) in deployment.

Randomized synchronous BFT. While randomized synchronous BFT protocols have expected $O(1)$ time, most solutions have not yet been practical. PiLi [42] achieves optimistic responsiveness and maintains robustness under a weaker network assumption named weak synchrony proposed in [68] and called mobile sluggish model in [12]. However, the latency for delivering a block is up to 40Δ to 60Δ . The multi-valued agreement protocol proposed by Abraham et al. [41] can be directly used as a BFT protocol achieving the 5Δ to 8Δ latency. As the only randomized synchronous BFT protocol being implemented, Dfinity consensus [29] suffers from unbounded communication complexity. Abraham et al. [45] fixed the flaw of Dfinity. Nevertheless, both protocols still have much lower performance in terms of both latency and throughput than deterministic BFT protocols, such as Sync

HotStuff, in failure-free scenarios. In particular, the concrete latency of Dfinity consensus is 8Δ . In contrast, Sonic achieves a lower latency of 2Δ .

BFT with multiple proposers. Many BFT protocols are designed with the multi-proposer feature. Protocols like Mir-BFT [69], RCC [70], ISS [71], SpotLess [72] run parallel BFT instances where each instance can be viewed as a leader-based BFT. Another line of work de-couples block proposals from agreement on the order. Multiple proposers can propose new blocks simultaneously and later agree on their order. Examples include Dumbo-NG [46], Star [20], and Autobahn [62]. In fact, directed acyclic graph (DAG)-based protocols can also be viewed as protocols that de-couple block proposals from agreement on the order [63], [73]–[77]. The only difference is that, due to the use of DAG-based data structure to track the dependencies of proposed blocks, the agreement on the order does not have to be executed by an explicit protocol instance.

Meanwhile, many asynchronous BFT primitives are designed with the multi-proposer feature. One notable example is the asynchronous common subset (ACS) family. In ACS, each replica holds an input and outputs a subset of the inputs proposed by all parties. It was originally designed for multiparty computation but later found to be a useful consensus primitive for systems such as blockchains. Practical constructions include PACE [17], FIN [18], and Speeding-Dumbo [47].

In synchronous networks, to our knowledge, Sonic is the only protocol that delivers $O(n)$ blocks per epoch. To achieve this property, Sonic adopts a different approach, where in each epoch, rather than delivering all the new blocks proposed in this epoch, Sonic selects a single replica and delivers all blocks it has proposed but not yet delivered.

VIII. CONCLUSION

We present a randomized synchronous BFT paradigm reducing BFT to a new primitive, MBA-SEV. We present Sonic and P-Sonic, efficient instantiations of protocols under the paradigm. In addition to the expected $O(1)$ time complexity, both Sonic and P-Sonic achieve low communication, $O(n)$ amortized message complexity per block proposal, and near-optimal latency under failure-free scenarios. Our experimental results show that Sonic and P-Sonic well outperform existing the state-of-the-art synchronous BFT protocol.

REFERENCES

- [1] M. Castro and B. Liskov, “Practical byzantine fault tolerance,” in *OSDI*, 1999, pp. 173–186.
- [2] A. Bessani, J. Sousa, and E. E. Alchieri, “State machine replication for the masses with bft-smart,” in *DSN*, 2014, pp. 355–362.
- [3] J. Sousa and A. Bessani, “From byzantine consensus to bft state machine replication: A latency-optimal transformation,” in *EDCC*, 2012, pp. 37–48.
- [4] M. Abd-El-Malek, G. R. Ganger, G. R. Goodson, M. K. Reiter, and J. J. Wylie, “Fault-scalable byzantine fault-tolerant services,” in *SOSP*, 2005, pp. 59–74.
- [5] J. Cowling, D. Myers, B. Liskov, R. Rodrigues, and L. Shriram, “Hq replication: a hybrid quorum protocol for byzantine fault tolerance,” in *OSDI*, 2006, pp. 177–190.
- [6] R. Kotla, L. Alvisi, M. Dahlin, A. Clement, and E. Wong, “Zyzzyva: Speculative byzantine fault tolerance,” *ACM Transactions on Computer Systems (TOCS)*, vol. 27, pp. 7:1–7:39, 2010.
- [7] P. Raykov, N. Schiper, and F. Pedone, “Byzantine fault-tolerance with commutative commands,” in *OPODIS*, 2011, pp. 329–342.
- [8] G. Golan Gueta, I. Abraham, S. Grossman, D. Malkhi, B. Pinkas, M. Reiter, D.-A. Seredinschi, O. Tamir, and A. Tomescu, “Sbft: A scalable and decentralized trust infrastructure,” in *DSN*, 2019, pp. 568–580.
- [9] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, “Hotstuff: Bft consensus with linearity and responsiveness,” in *PODC*, 2019, pp. 347–356.
- [10] D. S. Silva, R. Graczyk, J. Decouchant, M. Völz, and P. Esteves-Verissimo, “Threat adaptive byzantine fault tolerant state-machine replication,” in *SRDS*, 2021, pp. 78–87.
- [11] J. Decouchant, D. Kozhaya, V. Rahli, and J. Yu, “Damysus: streamlined bft consensus leveraging trusted components,” in *EuroSys*, 2022, p. 1–16.
- [12] I. Abraham, D. Malkhi, K. Nayak, L. Ren, and M. Yin, “Sync hotstuff: Simple and practical synchronous state machine replication,” in *S&P*, 2020, pp. 106–118.
- [13] A. Miller, Y. Xia, K. Croman, E. Shi, and D. Song, “The honey badger of bft protocols,” in *CCS*, 2016, pp. 31–42.
- [14] S. Duan, M. K. Reiter, and H. Zhang, “Beat: Asynchronous bft made practical,” in *CCS*, 2018, pp. 2028–2041.
- [15] C. Liu, S. Duan, and H. Zhang, “Epic: Efficient asynchronous bft with adaptive security,” in *DSN*, 2020, pp. 437–451.
- [16] S. Duan and H. Zhang, “Foundations of dynamic bft,” in *S&P*, 2022, pp. 1317–1334.
- [17] H. Zhang and S. Duan, “Pace: Fully parallelizable bft from reproposeable byzantine agreement,” in *CCS*, 2022, pp. 3151–3164.
- [18] S. Duan, X. Wang, and H. Zhang, “Fin: Practical signature-free asynchronous common subset in constant time,” in *CCS*, 2023, pp. 815–829.
- [19] H. Zhang, S. Duan, B. Zhao, and L. Zhu, “Waterbear: practical asynchronous bft matching security guarantees of partially synchronous bft,” in *USENIX Security*, 2023, pp. 5341–5357.
- [20] S. Duan, H. Zhang, X. Sui, B. Huang, C. Mu, G. Di, and X. Wang, “Dashing and star: Byzantine fault tolerance with weak certificates,” in *EuroSys*, 2024, pp. 250–264.
- [21] C. Cachin and M. Vukolić, “Blockchain consensus protocols in the wild,” in *DISC*, 2017, pp. 1:1–1:16.
- [22] X. Wang, S. Duan, J. Clavin, and H. Zhang, “Bft in blockchains: From protocols to use cases,” *ACM Computing Surveys (CSUR)*, vol. 54, pp. 209:1–209:37, 2022.
- [23] A. Bessani, E. Alchieri, J. Sousa, A. Oliveira, and F. Pedone, “From byzantine replication to blockchain: Consensus is only the beginning,” in *DSN*, 2020, pp. 424–436.
- [24] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, “Hyperledger fabric: a distributed operating system for permissioned blockchains,” in *EuroSys*, 2018, pp. 1–15.
- [25] J. Sousa, A. Bessani, and M. Vukolić, “A byzantine fault-tolerant ordering service for the hyperledger fabric blockchain platform,” in *DSN*, 2018, pp. 51–58.
- [26] V. Buterin and V. Griffith, “Casper the friendly finality gadget,” 2017, arXiv:1710.09437.
- [27] E. Buchman, J. Kwon, and Z. Milošević, “The latest gossip on bft consensus,” 2018, arXiv:1807.04938.
- [28] D. Cason, E. Fynn, N. Milošević, Z. Milošević, E. Buchman, and F. Pedone, “The design, architecture and performance of the tendermint blockchain network,” in *SRDS*, 2021, pp. 23–33.
- [29] T. Hanke, M. Movahedi, and D. Williams, “Dfinity technology overview series, consensus system,” 2018, arXiv:1805.04548.
- [30] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich, “Algorand: Scaling byzantine agreements for cryptocurrencies,” in *SOSP*, 2017, pp. 51–68.
- [31] M. Fitzi, “Generalized communication and security models in byzantine agreement,” Ph.D. dissertation, ETH Zurich, 2002.
- [32] I. Abraham, K. Nayak, L. Ren, and Z. Xiang, “Byzantine agreement, broadcast and state machine replication with near-optimal good-case latency,” 2020, arXiv:2003.13155.
- [33] N. Shrestha, I. Abraham, L. Ren, and K. Nayak, “On the optimality of optimistic responsiveness,” in *CCS*, 2020, pp. 839–857.

- [34] I. Abraham, K. Nayak, and N. Shrestha, “Optimal good-case latency for rotating leader synchronous bft,” in *OPODIS*, vol. 217, 2022, pp. 27:1–27:19.
- [35] D. Dolev and H. R. Strong, “Authenticated algorithms for byzantine agreement,” *SIAM Journal on Computing*, vol. 12, pp. 656–666, 1983.
- [36] A. Momose and L. Ren, “Optimal communication complexity of authenticated byzantine agreement,” in *DISC*, vol. 209, 2021, pp. 32:1–32:16.
- [37] M. Ben-Or, “Another advantage of free choice (extended abstract): Completely asynchronous agreement protocols,” in *PODC*, 1983, pp. 27–30.
- [38] P. Feldman and S. Micali, “An optimal probabilistic protocol for synchronous byzantine agreement,” *SIAM Journal on Computing*, vol. 26, pp. 873–933, 1997.
- [39] M. Fitzi and J. A. Garay, “Efficient player-optimal protocols for strong and differential consensus,” in *PODC*, 2003, pp. 211–220.
- [40] J. Katz and C.-Y. Koo, “On expected constant-round protocols for byzantine agreement,” in *CRYPTO*, 2006, pp. 445–462.
- [41] I. Abraham, S. Devadas, D. Dolev, K. Nayak, and L. Ren, “Synchronous byzantine agreement with expected $O(1)$ rounds, expected $O(n^2)$ communication, and optimal resilience,” in *FC*, 2019, pp. 320–334.
- [42] T.-H. H. Chan, R. Pass, and E. Shi, “Pili: An extremely simple synchronous blockchain,” 2018, cryptology ePrint Archive, Paper 2018/980.
- [43] E. Shi, “Streamlined blockchains: A simple and elegant approach (a tutorial and survey),” in *ASIACRYPT*, 2019, pp. 3–17.
- [44] B. Y. Chan and E. Shi, “Streamlet: Textbook streamlined blockchains,” 2020, cryptology ePrint Archive, Paper 2020/088.
- [45] I. Abraham, D. Malkhi, K. Nayak, and L. Ren, “Dfinity consensus, explored,” 2018, cryptology ePrint Archive, Paper 2018/1153.
- [46] Y. Gao, Y. Lu, Z. Lu, Q. Tang, J. Xu, and Z. Zhang, “Dumbo-ng: Fast asynchronous bft consensus with throughput-oblivious latency,” in *CCS*, 2022, pp. 1187–1201.
- [47] B. Guo, Y. Lu, Z. Lu, Q. Tang, J. Xu, and Z. Zhang, “Speeding dumbo: Pushing asynchronous bft closer to practice,” in *NDSS*, 2022.
- [48] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, “Secure and efficient asynchronous broadcast protocols,” in *CRYPTO*, 2001, pp. 524–541.
- [49] R. Pass and E. Shi, “Hybrid consensus: Efficient consensus in the permissionless model,” in *DISC*, vol. 91, 2017, pp. 39:1–39:16.
- [50] M. Pease, R. Shostak, and L. Lamport, “Reaching agreement in the presence of faults,” *Journal of the ACM (JACM)*, vol. 27, pp. 228–234, 1980.
- [51] L. Lamport, R. Shostak, and M. Pease, “The byzantine generals problem,” *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 4, pp. 382–401, 1982.
- [52] C. Cachin, R. Guerraoui, and L. Rodrigues, *Introduction to Reliable and Secure Distributed Programming*, 2nd ed. Springer Publishing Company, Incorporated, 2011.
- [53] V. King and J. Saia, “Breaking the $O(n^2)$ bit barrier: scalable byzantine agreement with an adaptive adversary,” *Journal of the ACM (JACM)*, vol. 58, pp. 18:1–18:24, 2011.
- [54] T. Crain, “Two more algorithms for randomized signature-free asynchronous binary byzantine consensus with $t < n/3$ and $O(n^2)$ messages and $O(1)$ round expected termination,” 2020, arXiv:2002.08765.
- [55] S. Das, T. Yurek, Z. Xiang, A. K. Miller, L. Kokoris-Kogias, and L. Ren, “Practical asynchronous distributed key generation,” in *S&P*, 2022, pp. 2518–2534.
- [56] C. Cachin, K. Kursawe, and V. Shoup, “Random oracles in constantino-ple: Practical asynchronous byzantine agreement using cryptography,” *Journal of Cryptology*, vol. 18, pp. 219–246, 2005.
- [57] A. Boldyreva, “Threshold signatures, multisignatures and blind signatures based on the gap-diffie-hellman-group signature scheme,” in *PKC*, 2003, pp. 31–46.
- [58] V. Shoup, “Practical threshold signatures,” in *EUROCRYPT*, 2000, pp. 207–220.
- [59] A. Mostéfaoui, H. Moumen, and M. Raynal, “Signature-free asynchronous binary byzantine consensus with $t < n/3$, $O(n^2)$ messages, and $O(1)$ expected time,” *Journal of the ACM (JACM)*, vol. 62, pp. 31:1–31:21, 2015.
- [60] N. Alhaddad, S. Duan, M. Varia, and H. Zhang, “Succinct erasure coding proof systems,” 2021, cryptology ePrint Archive, Paper 2021/1500.
- [61] A. Mostéfaoui and M. Raynal, “Signature-free asynchronous byzantine systems: from multivalued to binary consensus with $t < n/3$, $O(n^2)$ messages, and constant time,” *Acta Informatica*, vol. 54, no. 5, pp. 501–520, 2017.
- [62] N. Giridharan, F. Suri-Payer, I. Abraham, L. Alvisi, and N. Crooks, “Autobahn: Seamless high speed bft,” in *SOSP*, 2024, pp. 1–23.
- [63] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, “Narwhal and tusk: a dag-based mempool and efficient bft consensus,” in *EuroSys*, 2022, pp. 34–50.
- [64] I. Keidar, E. Kokoris-Kogias, O. Naor, and A. Spiegelman, “All you need is dag,” in *PODC*, 2021, pp. 165–175.
- [65] H. Wang, Q. You, and S. Duan, “Synchronous byzantine agreement with $O(n)$ messages and $O(1)$ expected time,” *IEEE Transactions on Information Forensics and Security (TIFS)*, vol. 20, pp. 338–349, 2025.
- [66] I. Abraham, K. Nayak, L. Ren, and Z. Xiang, “Brief announcement: Byzantine agreement, broadcast and state machine replication with optimal good-case latency,” in *DISC*, vol. 179, 2020, pp. 47:1–47:3.
- [67] N. Milošević, D. Cason, Z. Milošević, and F. Pedone, “How robust are synchronous consensus protocols?” in *OPODIS*, vol. 324, 2024, pp. 20:1–20:25.
- [68] Y. Guo, R. Pass, and E. Shi, “Synchronous, with a chance of partition tolerance,” in *CRYPTO*, 2019, pp. 499–529.
- [69] C. Stathakopoulou, T. David, M. Pavlovic, and M. Vukolić, “Mir-bft: High-throughput robust bft for decentralized networks,” 2021, arXiv:1906.05552.
- [70] S. Gupta, J. Hellings, and M. Sadoghi, “Rcc: Resilient concurrent consensus for high-throughput secure transaction processing,” in *ICDE*, 2021, pp. 1392–1403.
- [71] C. Stathakopoulou, M. Pavlovic, and M. Vukolić, “State machine replication scalability made simple,” in *EuroSys*, 2022, pp. 17–33.
- [72] D. Kang, S. Rahnema, J. Hellings, and M. Sadoghi, “Spotless: Concurrent rotational consensus made practical through rapid view synchronization,” in *ICDE*, 2024, pp. 1916–1929.
- [73] A. Spiegelman, N. Giridharan, A. Sonnino, and L. Kokoris-Kogias, “Bullshark: Dag bft protocols made practical,” in *CCS*, 2022, pp. 2705–2718.
- [74] K. Babel, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, A. Koshy, A. Sonnino, and M. Tian, “Mysticeti: Reaching the latency limits with uncertified dags,” in *NDSS*, 2025.
- [75] B. Arun, Z. Li, F. Suri-Payer, S. Das, and A. Spiegelman, “Shoal++: High throughput dag bft can be fast and robust!” in *NSDI*, 2025, pp. 813–826.
- [76] I. Keidar, O. Naor, O. Poupko, and E. Shapiro, “Cordial miners: Fast and efficient consensus for every eventuality,” in *DISC*, vol. 281, 2023, pp. 26:1–26:22.
- [77] D. Malkhi, C. Stathakopoulou, and M. Yin, “Bbca-chain: Low latency, high throughput bft consensus on a dag,” in *FC*, 2025, pp. 51–73.

APPENDIX A

PROOF OF SONIC-BA AND ITS VBA VARIANT

Lemma 1. *If all correct replicas ba -propose v in round r , then all correct replicas enter round $r + 1$ by setting iv_{r+1} to v .*

Proof. If all correct replicas ba -propose v in round r , each correct replica p_i sends $pre_vote_r(v, \pi_i)$ to all replicas. No correct replica is able to collect $f + 1$ $pre_vote_r(\bar{v}, *)$. Therefore, upon timeout of t_1 , each correct replica has only one value v and is able to obtain a valid signature π . Each correct replica will send $main_vote_r(v, \pi)$ to all replicas. Similarly, after t_2 times out, each correct replica will receive at least $f + 1$ $main_vote_r(v, \pi)$ (i.e., $vcount_r(v) \geq f + 1$). Therefore, each correct replica may ba -decide or not. In both cases, every correct replica enters the next round with v as iv_{r+1} . $\square$

Lemma 2. *If all correct replicas ba -propose v in round r , then all correct replicas set $iv_{r'}$ to v for every round $r' > r$.*

Proof. The lemma can be proved by induction. The base case follows that of Lemma 1. We now show the case for r' by proving that if all correct replicas propose v in round r' , then all correct replicas set $iv_{r'+1}$ to v .

In round r' , if all replicas have $iv_{r'}$ to v , all correct replicas send $\text{pre-vote}_{r'}(v, *)$ to all replicas. No correct replica is able to collect $f + 1$ $\text{pre-vote}_{r'}(\bar{v}, *)$. It is straightforward to see that this case is the same as that in the base case. $\square$

Theorem 3. (Validity) *If all correct replicas ba -propose v , then any correct replica that terminates ba -decides v .*

Proof. According to Lemma 1, we know every correct replica either directly terminates in round 1 or enters round 2 with iv_2 as v . We assume that a correct replica p_j ba -decides $\bar{v}$ in round r' where $r' \geq 2$ and prove the theorem by contradiction. If p_j ba -decides $\bar{v}$, it has received a valid pair of $\bar{v}$ and π , where π combines $f + 1$ partial signatures. In other words, at least one correct replica has sent $\text{pre-vote}_r(\bar{v}, *)$ to all replicas, a contradiction with Lemma 2. $\square$

Lemma 4. *After timeout of t_2 , every correct replica must have received the $\text{main-vote}_r()$ messages from all correct replicas.*

Proof. Each replica sends a $\text{main-vote}_r()$ message to all replicas after t_1 times out. Therefore, after another Δ time (i.e., after t_2 expires), every replica is able to receive $\text{main-vote}_r()$ messages from all correct replicas. $\square$

Lemma 5. *After timeout of t_2 , if a correct replica p_i only has $\{(v, \pi)\} = vset_r$ and $vcount_r(v) \geq f + 1$, another correct replica p_j has $\{(v', \pi')\} = vset_r$ and $vcount_r(v') \geq f + 1$, $v = v'$ and $\pi = \pi'$.*

Proof. According to Lemma 4, every correct replica receives the $\text{main-vote}_r()$ messages from all correct replicas. If replica p_i has only one pair (v, π) in its $vset_r$ and has $vcount_r(v) \geq f + 1$ after t_2 times out, the $f + 1$ correct replicas must have all sent $\text{main-vote}_r(v, \pi)$ or $\text{main-vote}_r(\perp, \perp)$ to all replicas. Furthermore, since p_i receives at least $f + 1$ $\text{main-vote}_r(v, \pi)$, at least one correct replica has sent $\text{main-vote}_r(v, \pi)$ to all replicas. Similarly, if p_j only has (v', π') in its $vset_r$ and has $vcount_r(v') \geq f + 1$, the $f + 1$ correct replicas must have all sent $\text{main-vote}_r(v', \pi')$ or $\text{main-vote}_r(\perp, \perp)$ to all replicas. Furthermore, at least one correct replica has sent $\text{main-vote}_r(v', \pi')$ to all replicas. Therefore, at least $f + 1$ replicas have sent $\text{pre-vote}_r(v, *)$ to all replicas and at least $f + 1$ replicas have sent $\text{pre-vote}_r(v', *)$ to all replicas, which needs $v = v'$ and $\pi = \pi'$. $\square$

Lemma 6. *If a correct replica ba -decides v in round r , any correct replica that enters round $r + 1$ and uses $iv_{r+1} = v$.*

Proof. If a correct replica ba -decides v in round r , it has only one pair (v, π) in its $vset_r$ and $vcount_r(v) \geq f + 1$. According to Lemma 5, we know that it is impossible for another correct replica to have $vset_r = \{(\bar{v}, \pi')\}$ and $vcount_r(\bar{v}) \geq f + 1$. Therefore, every correct replica either enters the same condition with $iv_{r+1} = v$, or uses the common coin value to enter next round, which is also v . $\square$

Theorem 7. (Agreement) *If a correct replica ba -decides v , then any correct replica that terminates ba -decides v .*

Proof. According to Lemma 6, if a correct replica ba -decides v in round r , every correct replica enters round $r + 1$ by setting iv_{r+1} to v . According to Lemma 2, we know that in round r' such that $r' > r + 1$, correct replicas will propose v , unless they terminate the protocol. Now assume that a correct replica terminates and ba -decides $\bar{v}$. The replica must have received at least one $\text{main-vote}_r(\bar{v}, \pi')$, where π combines $f + 1$ partial signatures from the $\text{pre-vote}_r()$ step. In other words, at least one correct replica p_i has sent $\text{pre-vote}_r(\bar{v}, \pi_i)$ to all replicas, a contradiction with Lemma 2. $\square$

Theorem 8. (Termination) *Every correct replica eventually ba -decides some value.*

Proof. The proof consists of two parts. First, each replica is able to enter the next round unless it terminates the protocol. Second, in each round r , an adversary cannot manipulate the iv_{r+1} values used by replicas to enter next round.

We begin with the first part. Following the protocol, every replica sets a timer $t_1 \leftarrow \Delta$. We assume a synchronous network. Therefore, before t_1 times out, every correct replica receives all the $\text{pre-vote}_r()$ messages from every correct replica. After t_1 times out, every replica sends a $\text{main-vote}_r()$ message to all replicas, starts t_2 , and updates $vset_r$ after receiving valid $\text{main-vote}_r()$ messages from other replicas. After t_2 times out, every correct replica will update iv_{r+1} and enter the next round (unless replicas terminate the protocol).

We now prove the second part. All correct replicas will send $\text{main-vote}_r()$ messages to all replicas after t_1 times out, and stop collecting $\text{main-vote}_r()$ messages after t_2 times out. At this point, it is guaranteed that replicas have all received $\text{main-vote}_r()$ messages from correct replicas. Therefore, even if the adversary learns the value of the common coin, it is not able to manipulate the output as replicas do not accept $\text{main-vote}_r()$ messages any more. $\square$

Theorem 9. (Integrity) *No correct replica ba -decides twice.*

Proof. Integrity holds by inspection of the protocol. $\square$

Proof of VBA version of Sonic-BA. For VBA version of Sonic-BA, the validity, agreement, termination, and integrity property follow that of original Sonic-BA. We now prove the external validity property.

Theorem 10. (External Validity) *Any correct replica that terminates and vba -decides $(1, (h, \pi))$ validated by π such that $Q(1, \pi)$ holds.*

Proof. Every replica verifies whether $Q(1, \pi)$ holds upon receiving a $\text{pre-vote}_r(1, *)$. If a replica vba -decides $(1, (h, \pi))$, it has received $f + 1$ $\text{main-vote}_r(v, *)$. In other words, at least one correct replica has received $f + 1$ $\text{pre-vote}_r(1, *)$. Therefore, $Q(1, \pi)$ holds. $\square$

APPENDIX B

COMPARISON OF SYNCHRONOUS BYZANTINE BINARY AGREEMENT PROTOCOLS

We compare our Sonic-BA with other synchronous BA

TABLE II

COMPARISON OF SYNCHRONOUS BYZANTINE BINARY AGREEMENT PROTOCOLS WITH OPTIMAL RESILIENCE. THE NUMBER OF STEPS AND COMPLEXITY OF KK-BA IS CALCULATED BY ASSUMING THAT THE COMMON COIN ORACLE COMPLETES IN ONE STEP WHILE THE ORIGINAL WORK USES A STANDALONE OBLIVIOUS LEADER ELECTION (OLE) PROTOCOL FROM VERIFIABLE SECRET SHARING (VSS). λ DENOTES THE SECURITY PARAMETER. THE x/y DENOTES THE NUMBER OF STEPS TO TERMINATE. x IS THE NUMBER OF STEPS TO TERMINATE UNDER THE STATIC ADVERSARY ASSUMPTION AND y IS THE NUMBER UNDER THE ADAPTIVE SECURITY ASSUMPTION.

protocol	resilience	message	word	comm.	time	exp. # steps	best # steps
KK-BA [40]	$n > 2f$	$O(n^2)$	$O(n^2)$	$O(\lambda n^2)$	$O(1)$	12	6
ADDNR [41]	$n > 2f$	$O(n^2)$	$O(n^2)$	$O(\lambda n^2)$	$O(1)$	11/17	7/10
Sonic-BA (this work)	$n > 2f$	$O(n^2)$	$O(n^2)$	$O(\lambda n^2)$	$O(1)$	5	3

protocols with optimal resilience in Table II. Here, we analyze the number of steps for Sonic-BA.

Efficiency. Sonic-BA consists of 2 steps in each round and 1 step for querying the common coin. In a static adversary model, the common coin can be obtained in parallel with the second step. Therefore, considering the terminating step, in the optimistic case, Sonic-BA terminates in 3 steps. As replicas are expected to decide in 2 rounds, they are expected to decide in 4 steps. Thus, Sonic-BA is expected to terminate in 5 steps. As replicas query the common coin upon receiving $f + 1$ pre-vote_r() messages, the rushing adversary cannot manipulate the values used by each correct replica. Therefore, in the rushing adversary model, due to parallelization between querying the common coin and sending main-vote_r() messages, the optimistic number of steps to terminate is still 3, and the expected number of steps to terminate is still 5. As before, replicas are expected to decide in 4 steps.

APPENDIX C

EXTENDING SONIC-BA TO MULTI-VALUED BYZANTINE AGREEMENT

We present an approach that transforms BA into MBA, motivated by prior asynchronous MVBA [48]. The idea is to first let replicas exchange their proposed values. Replicas then iterate a consensus loop until they decide. In each iteration, replicas query a random permutation to select a leader. A VBA is used to decide whether the value from the selected leader is received or not. The iteration continues until replicas *ba-decide* 1 in VBA.

Our MBA protocol Sonic-MBA terminates in 4 steps under static adversary and 5 steps under rushing adversary optimistically. Sonic-MBA is expected to terminate in 11 steps under static adversary and 12 steps under rushing adversary. Sonic-MBA has $O(n^2)$ message complexity and $O(Ln^2 + \lambda n^2)$ communication complexity (L denotes the message size).

Additionally, leveraging erasure coding proof systems [60], the communication complexity of Sonic-MBA can be optimized into $O(Ln + \lambda n^2)$.

The MBA transformation. We now present Sonic-MBA by reducing the MBA problem to VBA. The pseudocode and the workflow of Sonic-MBA are shown in Fig. 14 and Fig. 15, respectively. We define the predicate Q as follows: $Q(1, \pi)$ holds if π is a valid threshold signature for (r, h_k) , where r is the identifier for the VBA instance, and h_k is the hash of the message from the selected leader in iteration r .

```

00 initialization
01  $M \leftarrow [\perp]^n$  {vector of received values}
02 func mba-propose( $v_{input}$ )
03   start  $t_1 \leftarrow \Delta$ 
04    $\delta \leftarrow \text{sign}(v_{input})$ 
05   send input( $v_{input}, \delta$ ) to all replicas ▷ Phase 1: Input
06   upon receiving input( $v_j, \delta_j$ ) from  $p_j$ 
07      $M[j] \leftarrow (v_j, \delta_j)$ 
08   upon timeout( $t_1$ )
09      $r \leftarrow 0$ 
10   Choose a random permutation  $\Pi$  ▷ Phase 2: Permutation
      {included inside the loop under adaptive adversary}
11   repeat ▷ Phase 3: Iteration of consensus
12      $r \leftarrow r + 1$ 
13      $k \leftarrow \Pi(r)$ 
14      $V \leftarrow \emptyset$  { $V$  is the set of inconsistent values}
15     start  $t_2 \leftarrow \Delta$ 
16     if  $M[k] \neq \perp$ 
17       parse  $M[k]$  as  $(v_k, \delta_k)$ 
18        $\pi_i \leftarrow \text{tsign}(r, H(v_k))$ 
19       send distr( $M[k], \pi_i$ ) to all replicas
20     else send distr( $\perp, \perp$ ) to all replicas
21     upon receiving distr( $(v_k, \delta_k), \pi_j$ ) s.t. verify( $v_k, \delta_k$ )
22        $V.add(v_k)$ 
23     upon timeout( $t_2$ )
24       if there are  $n - f$  distr( $(v_k, \delta_k), \pi_j$ ) and  $\{v_k\} = V$ 
25          $\pi \leftarrow \text{tcomb}(\pi_j, \dots)$ 
26         vba-propose(1, ( $H(v_k), \pi$ )) for VBA instance  $r$ 
27       else vba-propose(0,  $\perp$ ) for VBA instance  $r$ 
28       if vba-decide(1, ( $h_k, \pi$ ))
29         obtain  $v_k$  s.t.  $v_k \in V$  and  $H(v_k) = h_k$ 
30         mba-decide( $v_k$ )

```

Fig. 14. Sonic-MBA: MBA transformed from VBA.

Each replica p_i starts a timer t_1 and sends a message *input*(v_{input}, δ_i) to all replicas, where v_{input} is the *mba-proposed* value and δ_i is a digital signature for v_{input} . Each replica maintains a vector M , keeping track of the values received from other replicas. After t_1 expires, replicas query the random permutation Π and enter the iteration phase, starting from iteration 1.

In each iteration r , replicas reach an agreement on the value sent by replica p_k selected by Π . Replicas first start a timer t_2 . If a replica has a non-empty $M[k] = (v_k, \delta_k)$, it creates a partial signature π_i for $(r, H(v_k))$, and sends a *dist_r*($M[k], \pi_i$) message to all replicas. For every received *dist_r*($M[k], \pi_j$) message with non-empty $M[k]$ consisting of v, δ , the replica adds the verified value v to a set V . After t_2 expires, if the replica receives $n - f$ *dist_r*() messages and only has one

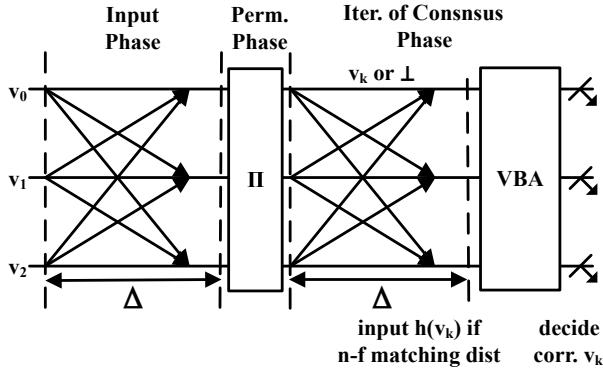


Fig. 15. The workflow of Sonic-MBA.

value in V , it combines the partial signatures into π and *vba-proposes* 1 along with its proof $(H(v_k), \pi)$. Otherwise, the replica *vba-proposes* 0. If the replica *vba-decides* the tuple $(1, (h_k, \pi))$, it obtains the original value from V and *mba-decides* this value. If the replica *vba-decides* 0, it continues the iteration and enters iteration $r + 1$.

Complexity. The transformation achieves $O(n^2)$ message complexity. Since the original message (sized L) is included in the first phase and signatures or hashes are included in other phases, the communication complexity is bounded by $O(Ln^2 + \lambda n^2)$.

Efficiency. We consider that the random permutation completes in one step, following prior works [41], [48]. In a static adversary model, the permutation phase can be executed in parallel with the input phase. Since, with a probability of $1/2$, a correct replica p_k is selected, the loop (distribution and VBA) is expected to end in 2 iterations, and the VBA is expected to terminate in 5 steps (and 3 steps in the optimistic case). Furthermore, replicas do not have to wait for the VBA to terminate before they *mba-decide*. Instead, each replica can *mba-decide* after it *vba-decides*. The optimistic number of steps to decide for VBA is 2 and the expected number of steps is 4.

If we assume static adversary, the permutation step and the input step can be run in parallel. Doing so is correct because the random permutation selects a sequence of random replicas and the corrupted replicas are already *fixed* before the protocol starts. Therefore, in the optimistic case, the Sonic-MBA protocol terminates in 4 steps (1 step for input and permutation, 1 step for sending $\text{dist}_r()$ messages, and 2 steps for VBA). The expected number of steps is $1 + 2 \times (1 + 4) = 11$ steps.

To handle rushing adversary, the permutation step has to be run after the input step. After the random permutation, the adversary cannot manipulate the results, as the messages each replica received (i.e., M) is *fixed* before the iterations of consensus. Therefore, the optimistic number of steps is 5 and the expected number of steps is $2 + 2 \times (1 + 4) = 12$. We compare our Sonic-MBA with other synchronous MBA protocols with optimal resilience in Table III.

Proof of Sonic-MBA. We prove the correctness of Sonic-MBA.

Theorem 11. (Validity): *If all replicas mba-propose v , any correct replica that terminates mba-decides v .*

Proof. If all replicas *mba-propose* v , all replicas will only have v in the M array. In other words, no matter which instance k is selected, every replica will only distribute v and similarly, *vba-decide* $(1, (H(v), \pi))$. $\square$

Lemma 12. *If a replica p_i (correct or not) vba-proposes $(1, (h_1, \pi))$, a replica p_j (correct or not) vba-proposes $(1, (h_2, \pi'))$, either $v_1 = v_2$ or all correct replicas vba-propose $(0, \perp)$.*

Proof. If p_i provides h_1 validated by π as input for VBA, it has received $f + 1 \text{ dist}_r((v_1, \delta_1), \pi_j)$ where $0 \leq j < n$ and $H(v_1) = h_1$. If p_j provides h_2 validated by π' as input for VBA, it has received $f + 1 \text{ dist}_r((v_2, \delta_2), \pi'_j)$ where $0 \leq j < n$ and $H(v_2) = h_2$. In other words, at least one correct replica has sent $\text{dist}_r((v_1, \delta_1), \pi_j)$ and at least one correct replica has sent $\text{dist}_r((v_2, \delta_2), \pi'_j)$. If $h_1 \neq h_2$, all correct replicas will receive both $\text{dist}_r((v_1, \delta_1), \pi_j)$ and $\text{dist}_r((v_2, \delta_2), \pi'_j)$, i.e., $V = \{v_1, v_2\}$. In this case, all correct replicas will *vba-propose* $(0, \perp)$. Therefore, either $h_1 = h_2$ or all correct replicas *vba-propose* $(0, \perp)$. $\square$

Theorem 13. (Agreement): *If a correct replica mba-decides v , then any correct replica that terminates mba-decides v .*

Proof. We assume that a correct replica p_j terminates and *mba-decides* v' and prove the theorem by contradiction. If a correct replica *mba-decides* v , it has *vba-decided* $(1, (H(v), \pi))$ in some round r . According to the agreement and termination properties of VBA, any correct replica p_j must have already *vba-decided* $(1, (H(v'), \pi'))$ in the same round r . We now show that $v = v'$.

According to Lemma 12, if $v \neq v'$, all correct replicas *vba-propose* $(0, \perp)$. According to the validity property of VBA, however, any correct replica that terminates *vba-decides* $(0, \perp)$, a contradiction. Therefore, $v = v'$. $\square$

Theorem 14. (Termination): *Every correct replica eventually mba-decides some value.*

Proof. According to the protocol, every replica sets the timer t_1 and t_2 . Upon time out, every replica sends a message regardless of its current state. Furthermore, the termination property of VBA guarantees that all replicas will eventually *vba-decide* some value.

We now show that eventually in some instance, all correct replicas *vba-decide* $(1, (h_k, \pi))$. If a sender p_k is correct, all correct replicas receive v_k, δ_k and set $M[k]$ to (v_k, δ_k) . Similarly, every correct replica will send $\text{dist}_r((v_k, \delta_k), \pi_i)$, receive at least $n - f \text{ dist}_r((v_k, \delta_k), \pi_j)$ from other replicas, and *vba-propose* $(1, (h_k, \pi))$. According to the validity property of VBA, all correct replicas will *vba-decide* $(1, (h_k, \pi))$. Since at least one correct replica has received $n - f \text{ dist}_r((v_k, \delta_k), \pi_j)$, at least one correct replica has sent a $\text{dist}_r()$ message to all replicas. Therefore, all correct replicas will have v_k in V .

TABLE III

COMPARISON OF SYNCHRONOUS MULTI-VALUED BYZANTINE AGREEMENT PROTOCOLS WITH OPTIMAL RESILIENCE. λ DENOTES THE SECURITY PARAMETER, AND L REPRESENTS THE LENGTH OF MULTI-VALUE INPUTS. THE x/y DENOTES THE NUMBER OF STEPS TO TERMINATE. x IS THE NUMBER OF STEPS TO TERMINATE UNDER THE STATIC ADVERSARY ASSUMPTION AND y IS THE NUMBER UNDER THE ADAPTIVE SECURITY ASSUMPTION.

protocol	resilience	message	word	communication	exp. # steps	best # steps
KK [40]	$n > 2f$	$O(n^2)$	$O(n^2)$	$O(Ln^2 + \lambda n^2)$	20	10
ADDNR [41]	$n > 2f$	$O(n^2)$	$O(n^2)$	$O(Ln^2 + \lambda n^2)$	11/17	7/10
Sonic-MBA (this work)	$n > 2f$	$O(n^2)$	$O(n^2)$	$O(Ln^2 + \lambda n^2)$	11/12	4/5

such that $H(v_k) = h_k$. In other words, all replicas *mba-decides* v_k . Note that with a probability of $(f+1)/(2f+1)$, a correct replica is selected by the random permutation Π . As $n = 2f+1$, every correct replica eventually *mba-decides* some value. The protocol completes in expected $O(1)$ time. $\square$

Theorem 15. (Integrity): No correct replica *mba-decides* twice.

Proof. The integrity property of VBA guarantees that every correct replica *vba-decides* once. Since every replica *mba-decides* after it *vba-decides*, integrity follows. $\square$

APPENDIX D

DEFINITION OF MULTI-VALUED VALIDATED BYZANTINE AGREEMENT (MVBA)

Cachin, Kursawe, Petzold, and Shoup (CKPS) [48] propose multi-valued validated Byzantine agreement (MVBA), called VBA in the original work. MVBA has a validity condition, *external validity*. In MVBA, each replica adds a global predicate Q known to all parties such that $Q(v, \pi)$ holds. In practice, such a validation proof can be constructed using, e.g., threshold signatures. An MVBA protocol with predicate Q should satisfy the following properties:

- **External validity:** Every correct replica that terminates *mvba-decides* v validated by π such that $Q(v, \pi)$ holds.
- **Agreement:** If a correct replica *mvba-decides* v , then any correct replica that terminates *mvba-decides* v .
- **Integrity:** If all replicas follow the protocol, and if a correct replica *mvba-decides* v validated by π , then some replica *mvba-proposed* v validated by π .
- **Termination:** If all correct replicas are activated and all messages sent among correct replicas have been delivered, then all correct replicas *mvba-decide*.

If a non-trivial predicate Q is used, MVBA rules out the situation where an adversary can easily propose an inconsistent value from correct replicas. Therefore, MVBA does not have to explicitly define validity. Our MBA with validity property can be directly transformed to MVBA by adding a predicate, which might be of independent interest.

APPENDIX E

PROOF OF SONIC-MBA-SEV

Theorem 16. (Unanimous validity): If all correct replicas *mba-propose* v and no replica can *mba-propose* v' such that both $Q(v, \delta)$ and $Q(v', \delta')$ hold for some δ and δ' , then any correct replica that terminates *mba-decides* v .

Proof. If all correct replicas *mba-propose* v , every correct replica provides a valid digital signature δ such that $Q(v, \delta)$ holds. According to the unforgeability of digital signatures, there does not exist a $\delta' \neq \delta$ such that $Q(v, \delta')$ also holds. Therefore, all correct replicas send $\text{input}(v, \pi_i, \delta)$ and all correct replicas will receive $f+1$ $\text{input}(v, \pi_j, \delta)$, add v to V . As no replica can *mba-propose* v' such that $Q(v', \delta')$ hold for some δ' , no correct replica will add any other value to V .

Meanwhile, as every correct replica receives $f+1$ $\text{input}(v, \pi_i, \delta)$ messages, it sets v_{mba} as v , obtain a threshold signature π_{mba} , and sets δ_{mba} as δ . No replica is able to provide a valid threshold signature for v' . Therefore, every correct replica will send $\text{dist}((v, \delta), \pi_{mba})$ to all replicas, and every correct replica has only one tuple in C . Then, every correct replica *vba-proposes* $(1, (H(v), \pi_{mba}))$. According to the validity property of VBA, all correct replicas will *vba-decide* $(1, (H(v), \pi_{mba}))$ and then *mba-decide* v .

Note that, in the fast path, the argument above also holds. $\square$

Lemma 17. In the slow (non-fast) path, if a correct replica *mba-decides* v , it receives a $\text{dist}((v, \delta), \pi)$ message such that $Q(v, \delta)$ holds and π is a threshold signature for v .

Proof. If a correct replica *mba-decides* v , it *vba-decides* $(1, (H(v), \pi))$, where π is a threshold signature for v . Therefore, at least one correct replica *vba-proposes* $(1, (H(v), \pi))$, as otherwise the validity property of VBA is violated. If a correct replica *vba-proposes* $(1, (H(v), \pi))$, it adds (v, π) to C . Before a correct replica adds (v, π) to C , it verifies whether $Q(v, \delta)$ holds, where δ is obtained from a $\text{dist}((v, \delta), \pi)$ message. $\square$

Theorem 18. (External validity): Every correct replica that terminates *mba-decides* v validated by δ such that $Q(v, \delta)$ holds.

Proof. According to Lemma 17, in the slow path, any correct replica that *mba-decides* v receives a $\text{dist}((v, \delta), \pi)$ such that $Q(v, \delta)$ holds. Therefore, the external validity property holds.

In the fast path, every correct replica *mba-decides* v upon receiving n $\text{input}(v, \delta, \pi_i)$ messages. Therefore, $Q(v, \delta)$ also holds, as all correct replicas *mba-propose* v . $\square$

Lemma 19. If a replica p_i (correct or not) *vba-proposes* $(1, (h_1, \pi))$, a replica p_j (correct or not) *vba-proposes* $(1, (h_2, \pi'))$, either $v_1 = v_2$ or all correct replicas *vba-propose* $(0, \perp)$.

Proof. If p_i *vba-proposes* $(1, (h_1, \pi))$, it has received $n - f$ $\text{dist}((v_1, \delta_1), \pi)$ messages and $H(v_1) = h_1$. If p_j *vba-proposes* $(1, (h_2, \pi'))$, it has received $n - f$ $\text{dist}((v_2, \delta_2), \pi')$ and $H(v_2) = h_2$. In other words, at least one correct replica has sent $\text{dist}((v_1, \delta_1), \pi)$ and at least one correct replica has sent $\text{dist}((v_2, \delta_2), \pi')$. If $h_1 \neq h_2$, all correct replicas will receive both $\text{dist}((v_1, \delta_1), \pi)$ and $\text{dist}((v_2, \delta_2), \pi')$, i.e., $|C| = 2$. In this case, all correct replicas will *vba-propose* $(0, \perp)$. Therefore, either $h_1 = h_2$ or all correct replicas *vba-propose* $(0, \perp)$. $\square$

Theorem 20. (Agreement): *If a correct replica mba-decides v , then any correct replica that terminates mba-decides v .*

Proof. We first show that agreement holds without the fast path. Then we show correctness under the fast path.

We begin with the case where fast path is not used. We assume that a correct replica p_j terminates and *mba-decides* v' and prove the theorem by contradiction. If a correct replica *mba-decides* v , it has *vba-decided* $(1, (H(v), \pi))$. According to the agreement and termination properties of VBA, any correct replica p_j must have already *vba-decided* $(1, (H(v'), \pi'))$. We now show that $v = v'$.

According to Lemma 19, if $v \neq v'$, all correct replicas *vba-propose* $(0, \perp)$. According to the validity property of VBA, however, any correct replica that terminates *vba-decides* $(0, \perp)$, a contradiction. Therefore, $v = v'$.

We now show that the above statement holds even with the fast path. According to the protocol, a replica *mba-decides* v in the fast path upon receiving n $\text{input}(v, *, *)$. Therefore, all correct replicas *mba-propose* v . According to the unanimous validity property, any correct replica that terminates *mba-decides* v . $\square$

Theorem 21. (Non-intrusion): *If a correct replica mba-decides v , v is mba-proposed by at least one correct replica.*

Proof. According to Lemma 17, any correct replica that *mba-decides* v receives a $\text{dist}((v, \delta), \pi)$ message, where π is a threshold signature for v . As π is obtained from the $\text{input}()$ step. At least $f + 1$ replicas have sent $\text{input}(v, \pi_i, \delta)$ messages. Among them, at least one replica is correct. The correct replica *mba-propose* v, δ .

Note that the property holds even under the fast path, under an inspection of the protocol. $\square$

Theorem 22. (Termination): *Every correct replica eventually mba-decides some value.*

Proof. We first show that termination holds without the fast path. Then show that the protocol with fast path also achieves termination.

According to the protocol, every replica sets the timer t_1 and t_2 . Upon time out, every correct replica sends a $\text{dist}()$ message regardless of its current state and starts VBA upon receiving $n - f$ $\text{dist}()$ messages. Furthermore, the termination property of VBA guarantees that if all replicas start VBA, they will eventually *vba-decide* some value. If a correct replica *vba-decides*, it *mba-decides*.

We now show that with the fast path, termination still holds. According to the protocol, a replica *mba-decides* v upon receiving n $\text{input}(v, *, *)$ messages. In this case, the replica still participates in the protocol. If it does not wait until *vba-deciding* any value before terminating the protocol, it receives n $\text{terminate}(v)$ messages. In such case, all correct replicas have already *mba-decided* v and terminate the protocol. $\square$

Theorem 23. (Integrity): *No correct replica mba-decides twice.*

Proof. The integrity property of VBA guarantees that every correct replica *vba-decides* once. Since every replica *mba-decides* after it *vba-decides*, integrity follows.

According to the protocol, if a replica *mba-decides* in the fast path, it still participates in the protocol but does not *mba-decide* again. The integrity still holds. $\square$

APPENDIX F

PROOF OF OUR RANDOMIZED SYNCHRONOUS BFT PARADIGM

Theorem 24. (Safety): *If a correct replica bft-delivers a block b_1 before another block b_2 , then no correct replica bft-delivers b_2 before bft-delivering b_1 .*

Proof. We first show that the safety holds when a correct replica p_i *bft-delivers* block proposal b_1 and b_2 in the same epoch e . We assume p_i delivers these block proposals in an aggregated block B , and the corresponding hash chain of B is h . Both b_1 and b_2 are in B and b_1 is arranged before b_2 . Thus, p_i *mba-decides* h . According to the agreement property of MBA-SEV primitive, all correct replicas *mba-decide* h . Due to the non-intrusion property of MBA-SEV, h is *mba-proposed* by at least one correct replica and the replica has the corresponding B . For each correct replica, if it does not have the aggregated block B , it will ask for all replicas for B . The replicas having B will send B to the replica issued the request. Therefore, all correct replicas obtain B and *bft-deliver* b_1 before b_2 .

We now show that if a correct replica p_i *bft-delivers* b_1 and b_2 in different epochs, the safety still holds. We assume that p_i *bft-delivers* b_1 in epoch e_1 and b_2 in epoch e_2 . Without generality, we assume $e_1 < e_2$. We assume p_i delivers the aggregated block B_1 in epoch e_1 and B_2 in epoch e_2 , and the corresponding hash chain is h_1 and h_2 respectively. Thus, p_i *mba-decides* h_1 in epoch e_1 and *mba-decides* h_2 in epoch e_2 . According to the agreement and integrity property of MBA-SEV, all correct replicas *mba-decide* h_1 in e_1 and h_2 in e_2 . Same as above, all correct replicas deliver all block proposals in B_1 in epoch e_1 and all block proposals in B_2 in epoch e_2 . Thus, all correct replicas *bft-deliver* b_1 in epoch e_1 before *bft-delivering* b_2 in epoch e_2 . $\square$

Theorem 25. (Liveness): *If a block b is bft-proposed to all replicas, then all correct replicas eventually bft-deliver b .*

Proof. We first show that all replicas will finish the current epoch and enter the next epoch. It follows directly from the termination property of MBA-SEV.

We now show that eventually all blocks *bft-proposed* will be *bft-delivered* in subsequent epochs. If a b is *bft-proposed* to all replicas, there must be a correct replica p_i receives b and adds b to C of p_i . After a limited number of epochs, p_i sends $\text{propose}(b)$ to all replicas. Then, all correct replicas append b to their $M[i]$. Note that the synchrony assumption ensures that for all correct replicas, blocks in the buffer according to a correct replica are the same in both content and order. Therefore, when p_i is selected by the random permutation Π , all correct replicas issue the same aggregated block B containing b with a valid signature and *mba-propose* the corresponding hash chain h . According to the unanimous validity and the external validity of MBA-SEV, all correct replicas *mba-decide* h . Since the probability of p_i being selected by Π is $1/n$, eventually p_i will be selected and h will be *mba-decided*. After that, all correct replicas will *bft-deliver* b . $\square$

APPENDIX G

THE RECOVERY MECHANISM

In our protocols, we assume that all correct replicas are online. In our protocols, replicas agree on the hash of the proposed blocks. In case some replicas have not received the corresponding proposed block given a hash, it needs to trigger the state transfer mechanism to catch up with other replicas. Meanwhile, correct replicas might go offline in practice and later recover from the crash. In this section, we briefly discuss the state transfer and recovery mechanism, assuming that at least one correct replica is online, and each delivered full blocks are maintained by at least one correct online replica.

State transfer to fetch a missing block. Due to the non-intrusion property of MBA-SEV, when a block is delivered by a correct replica, at least one correct replica has this block. To fetch the missing block, a replica p_i sends a request to all replicas. Upon receiving this request, any correct replica that holds the block sends the block to p_i . The non-collision property of the hash function ensures that p_i can obtain the block. This process finishes in 2Δ time.

Crash-and-recovery. There are multiple approaches for crash-and-recovery. If the model allows a replica to notify other replicas before it gets offline, the recovery mechanism can be made easier. In particular, every replica maintains a vector of n digital signatures, each from one replica. Let the vector be DS . Each signature in DS is on a hash of a block and the epoch number of the block, denoting each replica's highest block. Before a replica p_i goes offline, it creates a signature σ_i on the hash of its highest block, sets $DS[i]$ as σ_i , and sends DS to all replicas. Upon receiving DS , each replica updates its own DS based on the epoch numbers of the signatures, and then stores DS in its stable storage. p_i then participates in the protocol for one more epoch and then goes offline. When p_i recovers and becomes online, it triggers a recovery procedure and obtains $DS[i]$ from the replicas. Based on the signature, it then triggers the state transfer procedure to obtain its history of delivered blocks before it goes offline.